The College of Wooster
Open Works

Senior Independent Study Theses

2025

Development of Artificial Reverberation in C++

William A. Sieber
The College of Wooster, wsieber25@wooster.edu

Follow this and additional works at: https://openworks.wooster.edu/independentstudy

Recommended Citation
Sieber, William A., "Development of Artificial Reverberation in C++" (2025). *Senior Independent Study Theses.* Paper 11318.

This Senior Independent Study Thesis is brought to you by Open Works, a service of The College of Wooster Libraries. It has been accepted for inclusion in Senior Independent Study Theses by an authorized administrator of Open Works. For more information, please contact openworks@wooster.edu.

© Copyright 2025 William A. Sieber

Development of Artificial Reverberation in C++

Independent Study Thesis

Presented in Partial Fulfillment of the Requirements for the Degree Bachelor of Arts in Computer Science in the Department of Mathematical & Computational Sciences at The College of Wooster

by
William Sieber
The College of Wooster
2025

Advised by:

Daniel Palmer (Computer Science)

Greg Slawson (Music)

THE COLLEGE OF
WOOSTER

© 2025 by William Sieber

Abstract

Artificial reverberation is a method of adding "echo" to a particular sound to achieve a desired effect. In doing so, a signal with a short impulse can sound as if it is played from a large reverberant space, such as a concert hall. Common C++ implementations of this effect usually fall under two different categories: the first using purely artificial means, and the second mimicking a provided impulse response. The former creates this effect via a number of delay lines in parallel, resulting in a reverberated sound provided a sufficiently high echo density is created. The latter uses the impulse response of an acoustic space to generate a realistic sounding reverb, which is added to the input signal. This thesis studies the practical and sonic differences between these algorithms and their implementations. This includes the recording and processing of impulse responses in an acoustic space. The final program will allow producers to use either one of these algorithms independently in the Digital Audio Workstation of their choice.

I shall be telling this with a sigh

Somewhere ages and ages hence:

Two roads diverged in a wood, and I -

I took the one less traveled by,

And that has made all the difference.

Robert Frost

Acknowledgments

This thesis would not have been possible without the help and guidance of many people. First, I would like to thank my advisors Dr. Palmer and Greg Slawson for being patient with me during the creation of this thesis. Through the countless false starts and dead ends, they helped guide me and motivate me when I needed it most. In a similar vein, I would also like to thank Melissa Puster, Dan Krites, Dr. John Musgrave and Paul Musgrave for their help obtaining information and answering the (many) questions I had. Without them, this thesis would not be complete.

I would like to thank my academic advisor Dr. Cara Haxo for providing a listening ear whenever I needed it. She has helped me academically and emotionally throughout my time at Wooster, and it is her guidance that has allowed me to make the most out of my time in college. I would also likewise like to thank Dr. Heather Guarnera for her guidance and support during my Junior IS; it is with her help that I was able to see the light at the end of the tunnel for the first time.

I would like to thank Grace and Lina for the time we spent together working (and struggling) through IS. While IS can be isolating at times, you both provided an outlet for me to share my latest interests with. You helped make the journey enjoyable, and motivated me to see things through to the very end.

I would similarly like to thank my friends here and back at home for giving me motivation to see this project through. To Mom, Dad, Grandma & Grandpa, Ethan, Jacob, CJ, Emma, Lee, Henry, Elena, Ethan & Anthony, and to Mikey and friends, you have helped me in more ways than you can imagine.

Most importantly, I would like to thank my partner Lily for their neverending support, love, and understanding as I have worked on this project. Through the long days and late nights, they have stuck by my side and provided more support than I could ever ask or wish to receive. You are the love of my life, and I cannot wait to spend the rest of my life with you.

Contents

Abstract iii

Dedication iv

Acknowledgments v

Contents vii

List of Figures ix

List of Tables xi

List of Listings xii

CHAPTER PAGE

1 Introduction 1

2 Background 3
- 2.1 What is Reverb? . . . 3
- 2.2 Historical Developments . . . 5
- 2.3 How is Reverb Used Today? . . . 6

3 Real-time Audio Programming 9
- 3.1 Sound As a Discrete Signal . . . 9
- 3.2 The Real-time Programming Bus Schedule . . . 12
- 3.3 Program Structure & Libraries . . . 16

4 Artificial Reverberation Design 18
- 4.1 Filters & Signal Flow Graphs . . . 18
- 4.2 The Schroeder Comb Filter Approach . . . 23
- 4.3 Convolution . . . 24

5 Real Acoustic Spaces 27
- 5.1 Mathematical Background . . . 27
- 5.2 Measuring the Impulse Response . . . 29
- 5.3 Results of a Sinusoid Sweep . . . 30

6 The Software 33
- 6.1 Data Structures and Implementation . . . 33
- 6.2 User Interface . . . 39
- 6.3 Compatibility with DAWs . . . 40

7 Results 41
7.1 Evaluating the Echo Density 41
7.2 Comparisons With a Real Acoustic Space 42
8 Conclusion and Future Work 43
APPENDIX PAGE
A Code Classes 45
References 63
Index 67

List of Figures

Figure Page

2.1 A waveform of a snare drum. 4
2.2 A waveform of a snare drum with reverberation. 4
2.3 A model of a Spring Reverberator [6]. 6
2.4 A screenshot of Reaper - a Digital Audio Workstation. 7

3.1 A graph of the wave $f(t) = cos(3t)$. 10
3.2 A graph of a waveform, deconstructed to its sinusoids. 11
3.3 A discrete graph of a waveform taken with 80 samples. 12
3.4 A discrete graph distinguishing buffers at regular intervals. 14
3.5 A discrete graph with a dropped buffer. 14
3.6 The flow of information for audio data in a digital system. 15
3.7 A Model of an Audio Plugin Application. 16
3.8 UML diagram of the Audio Plugin. 17

4.1 An example of a Fourier Transform taken from a bandlimited function. . . . 19
4.2 The Amplitude Response of a simple lowpass filter. 20
4.3 The Signal Flow Graph of a Comb Filter [29]. 21
4.4 The Amplitude Response of a Comb Filter. 22
4.5 The Signal Flow Graph of an Allpass Filter [28]. 22
4.6 A Signal Flow Graph of Schroeder's Reverberator using Comb Filters and Allpass Filters [28]. 24
4.7 A visual representation of two waveforms being convolved [23]. 25

5.1 A graph of (a) the Logarithmic Sinusoid Sweep and (b) its inverse. 30
(a) Logarithmic Sinusoid Sweep $x(t)$. 30
(b) Inverse filter $f(t)$ from $x(t)$. 30
5.2 A screenshot of the Impulse Response of Gault via Reaverb. 31
5.3 A screenshot of the Impulse Response as a function of amplitude and time. . 31
5.4 The IR normalized. 32
5.5 The IR normalized and filtered. 32
5.6 The IR as a function of (a) its amplitude with respect to time and (b) its dB with respect to time. 32
(a) The IR cropped. 32
(b) The IR with regard to dB. 32

6.1 The reverb plugin running within Reaper. 40

7.1 Several waveforms with a number of Comb Filters in parallel. 42
7.2 A waveform of a reverberated snare drum from a Convolved IR. 42

List of Tables

Table Page

5.1 Relevant Absorption Coefficients [33]. 29

List of Listings

Listing		Page
6.1	The high level code of a Comb Filter.	34
6.2	Code to calculate g within a delay line.	35
6.3	Code to take the sum of two signals.	36
6.4	Code to implement circular behavior in an array.	36
6.5	Code to process a buffer via Convolution.	38
6.6	Code to initialize values to process.	38
6.7	Code to create a UI element.	39
6.8	Code to update backend parameters.	40
A.1	DelayLine.h	45
A.2	DelayLine.cpp	46
A.3	TestReverb.h	48
A.4	TestReverb.cpp	48
A.5	PluginProcessor.h	51
A.6	PluginProcessor.cpp	52
A.7	PluginEditor.h	58
A.8	PluginEditor.cpp	59

CHAPTER 1

Introduction

Meaningful songs take the listener on a journey. Regardless of genre, music has the ability to connect people together of a wide variety of backgrounds and experiences. In one way, a composer chooses different notes in a piece to follow particular harmonic and melodic conventions. Meeting these conventions and subverting them in interesting ways is part of what makes music, music. In another way, much like a song's chord progression, the timbre of a sound itself can create greater depth and immersion in one's production to help bring the listener from point "A" to point "B". Several tools are at a producer's disposal to modify the color and shape of a particular sound in real time. For example, distortion can be added to a guitar, or delay can be added to a singer's vocals. Reverb is another such essential tool at the producer's disposal used in music production today.

The study of artificial reverberation goes back over a hundred years. During this time, the methods used to artificially reverberate sound has shifted from ad hoc methods to carefully crafted tools and programs. Today, the majority of programs are created for use under specific host applications. By easily handling input and output of other programs which manipulate a given signal, one can perform complex sound design by utilizing a combination of several programs in series.

What follows is a study into the development and execution of artificial reverberation in C++. To better understand how reverberation is used in the context of music production, we will begin with a historical view of this tool. Next, we will look at the programs and

software that producers use to create music today, and in what ways they use reverberation. Specifically, this will include discussion of Digital Audio Workstations and their associated programs. After, we will consider the digital signal processing theory required to create these tools. This will include how to represent sound in a digital manner and in what ways the computer processes this sound. Finally, we will discuss various methods of persisting sound with a short impulse. Someone inexperienced in this topic should not only gain an understanding in how artificial reverberators are made, but also be able to implement one themselves.

CHAPTER 2

Background

This chapter will provide the backgorund information necessary to understand what artificial reverberators are, why their use is important, and the different ways they are used by musicians today. It will begin with a broad definition of "reverberation" and discuss how sound persists in real-world spaces. After, it will discuss various methods of reverberating sound. Lastly, it will discuss how reverb is used today.

2.1 What is Reverb?

Imagine yourself speaking in a large concert hall. The audience does not only hear your voice; they hear the combination of both your voice and the reflections of your voice as they bounce off the walls and stage you are speaking on. In the right context, they serve both an artistic and practical purpose by enhancing the qualities of the sound being made on stage. In a broad sense, reverb can be defined as "an ambient space in the perception of the listener" [5]. This can take many different forms. Sound waves bouncing off the walls of a room, sound waves traveling through springs, sound waves traveling through a computer program - these are a few such ways in which a perceived ambient space can be created. It provides a texture that is considered "wet" and washes out the sound of an otherwise clear tone. In small amounts, it can be tasteful and enhance the timbre of

an instrument. In large amounts, it can overpower the initial tone and can disrupt other elements of the music, such as rhythm [1].

Consider an instrument with a short impulse response:

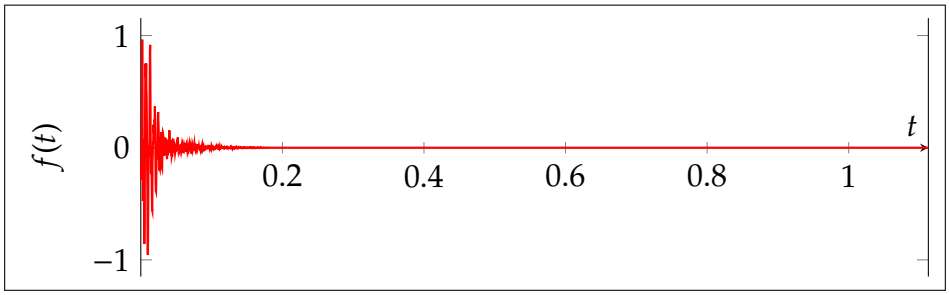


Figure 2.1: A waveform of a snare drum.

By adding reverb, this same instrument can sound as if it were performed in a large room:

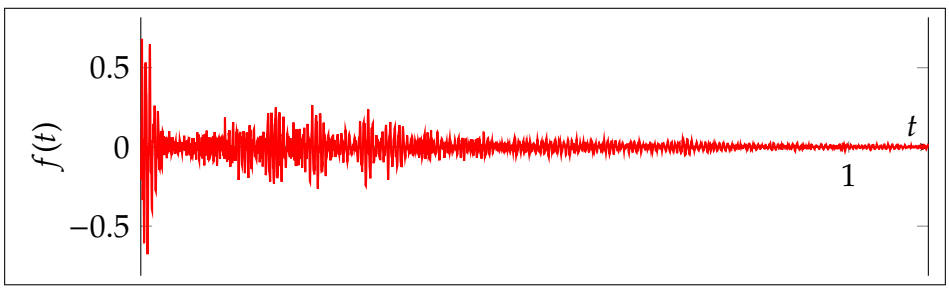


Figure 2.2: A waveform of a snare drum with reverberation.

Prior to the early 1960s, *artificial reverberators* were physical devices - that is, something that took input from an electromechanical transducer and manipulated the sound using a variety of physical methods. Some such devices used metal plates, springs, and oil canisters - each providing their own unique sound and characteristics [35]. In some cases, these sounds were desirable and used for their specific timbral qualities. Some programs today aim to replicate the sound of these early devices. Many, however, prefer to mimic the sound that would be expected of a large hall, without any of the uniqueness of other methods. In 1961, M. R. Schroeder introduced a number of methods to digitally recreate

reverb without additional color [28]. These same algorithms are used in several programs today.

2.2 Historical Developments

Prior to purely electronic means, there existed several methods of persisting sound with a short impulse. One such method is simply using a physical room. *Echo chambers* are rooms constructed lacking parallel surfaces to effectively reverberate a sound [35]. They contain a speaker and a microphone to record the input signal once it has been reverberated, which is then mixed back with the original signal. Their primary benefit is derived from the method itself - using a real room will produce a sound that is indistinguishable to an otherwise reverberant hall. While echo chambers continue to exist today, their use is limited to those who have the infrastructure to build and maintain such rooms. They are most commonly found in recording studios, or otherwise special music production spaces. For individual devices, there are other methods of reverberating sound that are more convenient and cost effective.

Spring Reverberators are another method. Their primary benefit is that they were compact - a much more convenient solution than an echo chamber. They were invented by Laurens Hammond after he patented the invention in 1941 [17]. In said patent, he describes a device in which sound traveling through a wire is sent through springs reinforced to the device, creating a reverberant sound. This reverberated signal is then sent back to be mixed with the original. Their small footprint allowed for their inclusion in devices such as synthesizers, a diagram of which can be seen below in Figure 2.3.

Electromechanical devices such as these are not perfect, however. Their sound may provide an approximation of what might be heard in an echo chamber, but is not a perfect replication of the original. *Plate reverberators* fall under a similar vein - sound travels through the plate and is mixed back with the original "dry" signal.

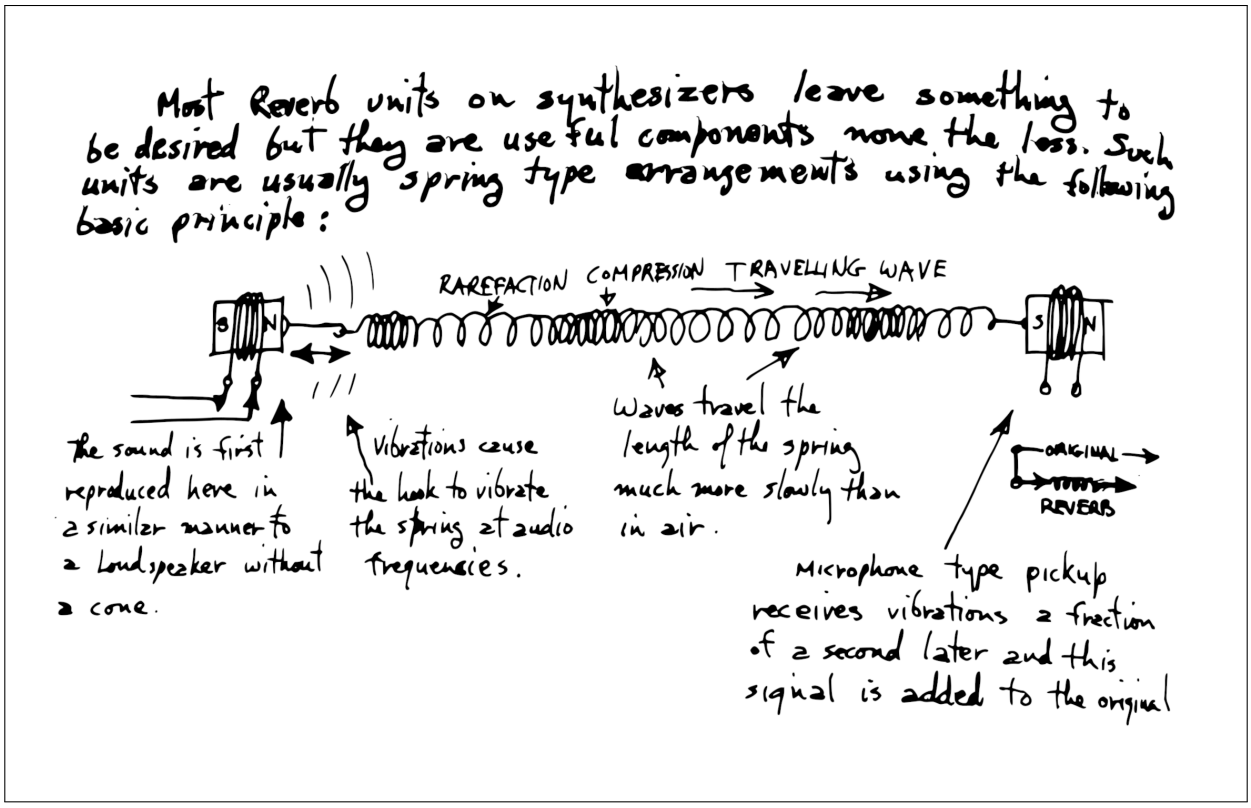


Figure 2.3: A model of a Spring Reverberator [6].

2.3 How is Reverb Used Today?

Nowadays, nearly every song one hears on the radio utilizes some kind of music application which allows one to record, edit, and manipulate several audio sources and recordings. Such applications are known as *Digital Audio Workstations* (DAWs). By allowing the producer to manipulate recordings in real time, they can quickly and efficiently create the music that one hears today. DAWs accomplish this by providing tools to record the notes or sounds of instruments through various means. For example, virtual instruments stored on the producer's PC can be performed by a MIDI keyboard conneced to the DAW. These virtual instruments are known as *plugins*, a type of application that runs under a DAW.

Plugins are generally broken into two distinct categories: *generator plugins* and *effect plugins*. Their difference is self-explanatory; generator plugins sample or synthesize a sound within the application, while effect plugins manipulate a given input. For the

purposes of this thesis, focus will be placed on the development of *effect plugins*. The producer can apply several of these effect plugins to a source sound in series. Effect plugins can range from distortion to chorus to arpeggiators, but they all operate using the same principle - take a signal as input, perform some type of operation to it, and return the processed signal. Reverb is one type of effect plugin that is often used in DAWs; an example of one such DAW can be seen in Figure 2.4.

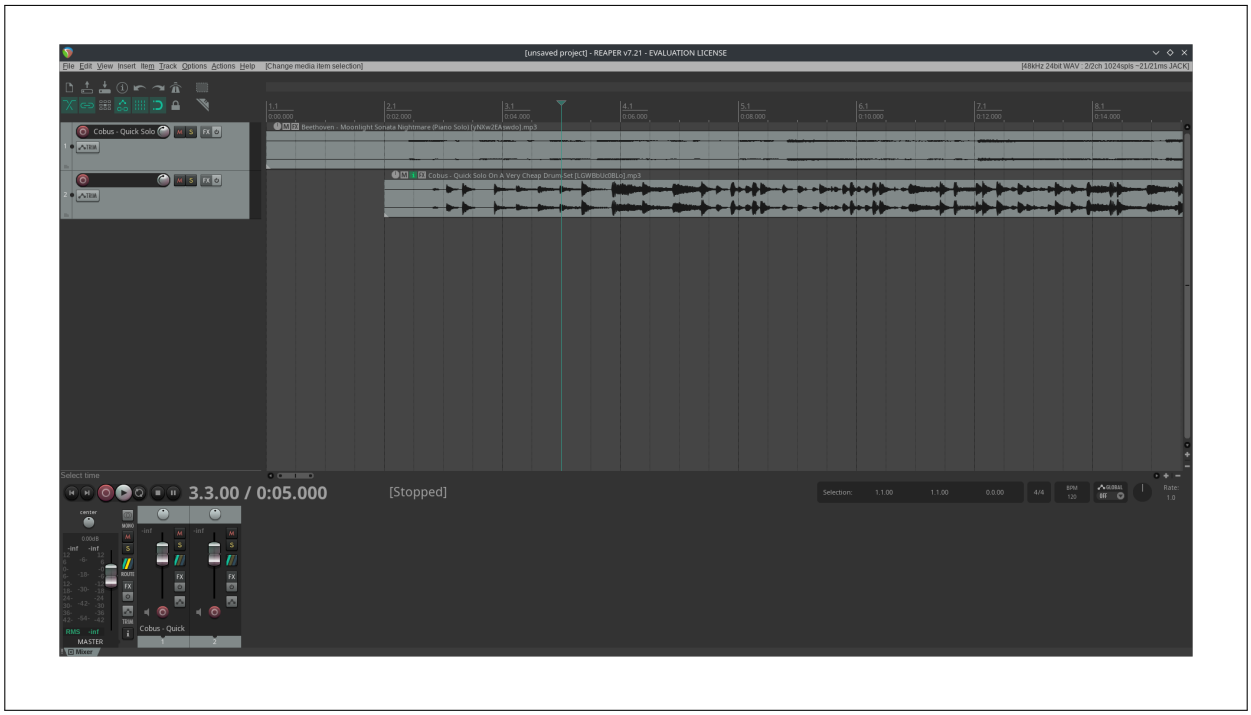


Figure 2.4: A screenshot of Reaper - a Digital Audio Workstation.

While DAWs can differ in look and feel, they are generally broken into four separate areas:

- The timeline: located in the middle, this contains each waveform and allows the producer to arrange how and when tracks are played.
- The track control panel: located on the left, this allows the producer to add and remove different tracks in a project.

- The mixer control panel: located on the bottom, this allows the producer to adjust the volume of individual tracks and add effects.

- The file explorer: not listed in the screenshot but commonly found, this allows the producer to browse samples and presets both included in the DAW and from their own filesystem.

These four sections are included in nearly every DAW available today. In practice, they allow producers to change when sounds occur and their volume by an exact amount. Through these tools, producers are able to create nearly any sound they can imagine; they can take parts of other songs and deconstruct them into something new; they can create an effect chain so complex that they turn a simple snare drum into something unrecognizable. The only limit is their creativity.

CHAPTER 3

Real-time Audio Programming

This chapter will consider the theory behind real-time audio programming and the different principles that are followed to create a functional program. It will begin with an overview of digital signal processing. Specifically, this will include the mathematical background required to understand sound. Next, it will cover the methods by which a digital system communicates with several components to create sound. Specifically, this will include how audio data is handled by a digital system's drivers and its corresponding speakers. After, the requirements that a real-time audio program needs in order to function will be considered. This will include a program's typical structure and functionality. Finally, libraries and other software available that handles these requirements will be discussed in length.

3.1 Sound As a Discrete Signal

Sound is a physical phenomenon. Vibrations travel through a medium such as air to create waves and differences in pressure, which travel at the speed of sound (roughly 331 m/s) [18]. These vibrations have varying lengths and intensity depending on the manner in which the sound is created. Changes in frequency are reflected as the pitch of a tone, such as different notes on an instrument. Changes in intensity reflect how loud a particular sound is, such as the different dynamics one can play on an instrument [18]. The human

brain interprets these differences in pressure by converting waves into electrical signals. When a wave reaches the human ear, the ear canal resonates and converts the sound into mechanical vibrations for the cochlea [18]. The cochlea in turn converts these into electrical signals for the brain. Like the human brain, interpreting sound in a digital setting involves simplifying the physical nature of sound waves into a format understandable by computers.

Consider Figure 3.1.

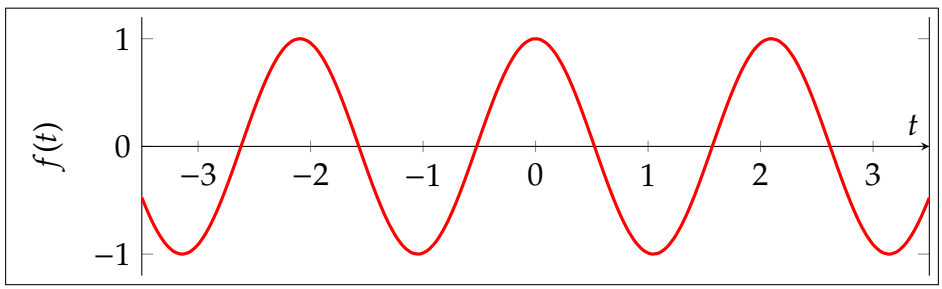


Figure 3.1: A graph of the wave $f(t) = cos(3t)$.

This is a representation of a sinusoid under the time domain. Each crest and trough represent the compressions and rarefactions that occur in real life when this tone is created. By graphing the sinusoid with respect to time, one can view the particular waveform that is created from this given sound. This is an example of a *continuous sinusoidal signal*. They are defined by the following expression:

Definition 3.1 Definition of a continuous sinusoid:

$$f(t) = Acos(2\pi ft + \Theta), \quad t \in (-\infty, \infty) \tag{3.1}$$

where f is frequency, t is time, and Θ is the phase offset in radians [34]. By taking the sum of several sinusoids, one can create (or deconstruct) any type of non-sinusoidal signal, given a sufficiently high number of sine waves are used. This is defined as:

Definition 3.2 Definition of a non-sinusoidal continuous signal:

$$f(t) = \sum_{k=1}^{N_h} A_k cos(2\pi k f t + \Theta_k) \tag{3.2}$$

where k is one sinusoid in the sum, N_h is the number of total sinusoids used to construct the signal, A_k the amplitude of a particular signal for each k, Θ_k the phase for each k, and f the fundamental frequency of $f(t)$ [34]. With this definition in mind, any signal - and therefore, any sound - can be represented as the sum of several sinusoids, seen in Figure 3.2.

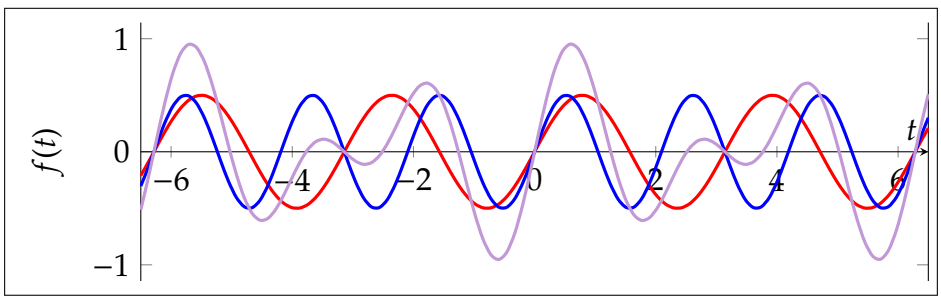


Figure 3.2: A graph of a waveform, deconstructed to its sinusoids.

To create a digitial representation of this signal, the *Nyquist-Shannon Sampling Theorem* provides guidelines for how signals should be *bandlimited* - or rather, have their frequencies limited - and how often a signal should be sampled to ensure its accuracy.

Definition 3.3 The Nyquist-Shannon Sampling Theorem [22]:

1. *The signal must be bandlimited, containing no frequencies beyond a particular maximum F_{max}.*

2. *The sampling rate of a signal must be at least twice the maximum frequency F_{max}.*

The human ear provides sufficient guidelines for each of these requirements. Typical human hearing spans across a range of 20 Hz - 20,000 Hz [18]. As such, the *Nyquist rate* - the minimum sampling rate - for a signal corresponding to an audio waveform must

encompass this range. It is defined as $F_s = 2F_{max}$ [22]. With an $F_{max} = 20,000$ Hz, the Nyquist rate for audio DSP applications is thus 40,000 Hz. This explains the sample rate for several audio applications. Modern audio recordings use 44,100 Hz, and Digital Audio Workstations usually produce with 48,000 Hz. With this Nyquist rate, the entire range of human hearing is covered.

The process of *sampling* an analog signal is done by taking measurements of a particular function at regular points in time. Each individual measurement of the original analog signal is a *sample*. By convention, samples are in the range (-1,1). This is in alignment with 32 bit wave files being of a *floating-point type*, as opposed to 16 bit wave files represented as *integers* the range (-32,768, 32,767).

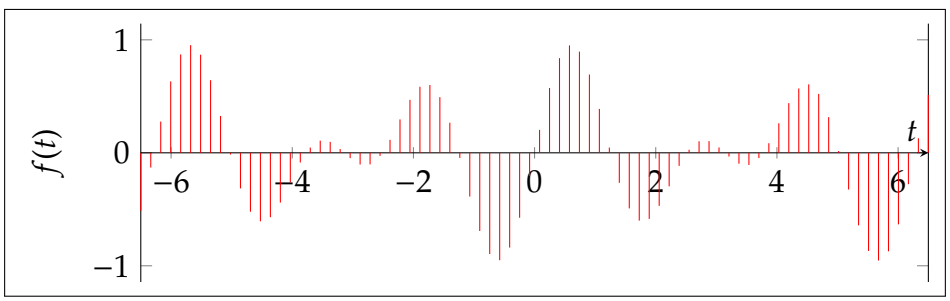


Figure 3.3: A discrete graph of a waveform taken with 80 samples.

This is a discrete representation of our original continuous signal. Such a snippet above represents a *buffer* of data. If each sample refers to a *float* in the range (-1,1), an array of floats is sufficient to store this waveform in memory. With this representation of sound, one can now use this data to process sound digitally.

3.2 The Real-time Programming Bus Schedule

In a digital system, hardware components get the attention of the processor by sending *interrupts* [26]. Keyboards, hard drives, and network cards are all components that use interrupts. By using interrupts, one can account for real-world imperfections in the transfer

of data. Speakers, likewise, utilize interrupts for the playback of sound. To play sound, a continuous stream of audio data must be sent to the sound card or audio chipset of the device [37]. To do this, data is sent in chunks, each known as a *buffer*. When the sound card finishes playing a buffer, it sends an interrupt to the processor requesting a new one for playback. For most applications, buffer sizes can be large enough to prevent audio from ever dropping. This allows for intermediate applications or virtual mixers to play audio from multiple sources without issue[1]. However, in the case that lower latency is expected for the end user, shortening buffer sizes can do so at the cost of higher performance requirements [4]. This high performant expectation is an example of a *real-time system*, which is defined as follows:

Definition 3.4 Definition of a Real-time System [3]:
A real-time system is one in which the correctness of the computations not only depends upon the logical correctness of the computation but also upon the time at which the result is produced.

Extended for DSP applications:

In a real-time DSP process, the analyzed (input) and/or generated (output) samples (whether they are grouped together in large segments or processed individually) can be processed (or generated) continuously in the time it takes to input and/or output the same set of samples independent of the processing delay.

Suppose, then, that a hardware device or software application interrupts the processor in the same manner during the time a high performant application is expected. To prevent low-priority interrupts from being executed before high-priority calls, *preemption* allows the processor to prioritize interrupts to maintain correctness in a real-time system [14]. On UNIX-like operating systems, interrupts are queued to be executed by the processor with

[1]This is a supposition based on observations made on PC audio - different operating systems and audio drivers likely handle things in different ways. On Windows, for example, if a buffer is about to be dropped (i.e., an interrupt is sent prior to the buffer being calculated), then the previous buffer is simply repeated by the audio driver.

a priority number. They are then dequeued in priority order, thus allowing applications with real-time requirements the ability to be prioritized [10]. While real-time systems can differ in how integral it is that deadlines are met, the audio stream can continue playback even if buffers are dropped. This is an example of a *soft real-time system*, as deadlines only need to be maintained *on average* [14]. In an ideal world, however, no buffers should be dropped to maintain integrity of the audio stream.

Consider Figure 3.4:

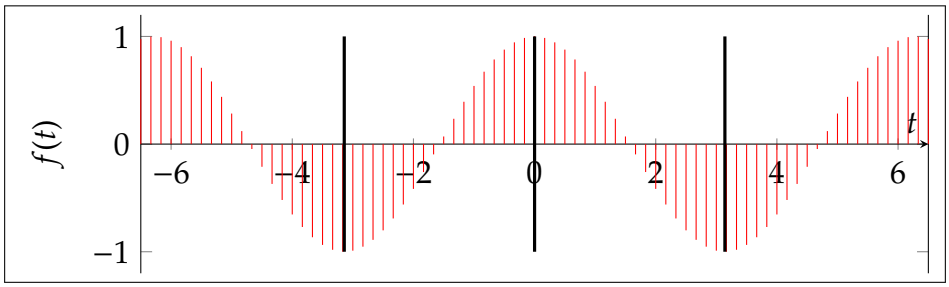


Figure 3.4: A discrete graph distinguishing buffers at regular intervals.

If a buffer is dropped in the digital system, it cannot be output by the speaker. The resulting sound will "click" and "pop" where there is a gap in the audio stream [37].

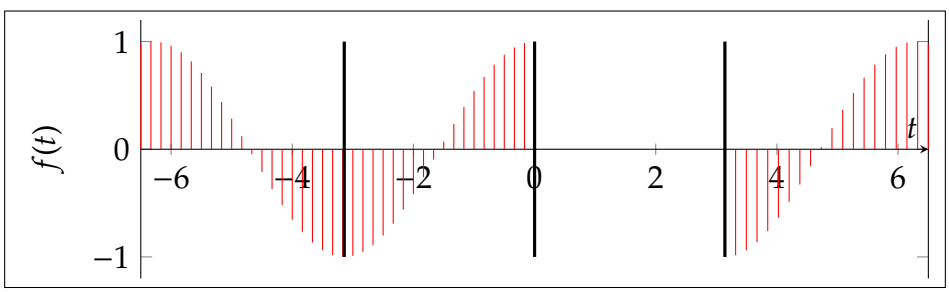


Figure 3.5: A discrete graph with a dropped buffer.

To prevent this from occuring, processing must occur within the time it takes to output a single buffer. The maximum time to process buffers is dependent of the buffer size. Suppose a single buffer is measured by an amount, x, which corresponds to the number of samples present in a single buffer of audio data. To obtain the amount of time t that a digital system would take to output this buffer, x can be divided by the *sample rate*,

measured in *hertz*. This measurement of hertz is by definition the reciprocal second. Thus, dividing the buffer size by the sample rate provides the time in seconds that the buffer takes to be output. For a buffer size x of 256 samples and a *sample rate* of 48,000 Hz, the resulting time t would be $(256/48,000)$, or 0.0053 seconds. Any such program working with real-time audio must compute a buffer during this time frame. Should processing take too long, audio will be dropped as seen in Figure 3.5.

To summarize, Figure 3.6 describes how this data travels in a digital setting.

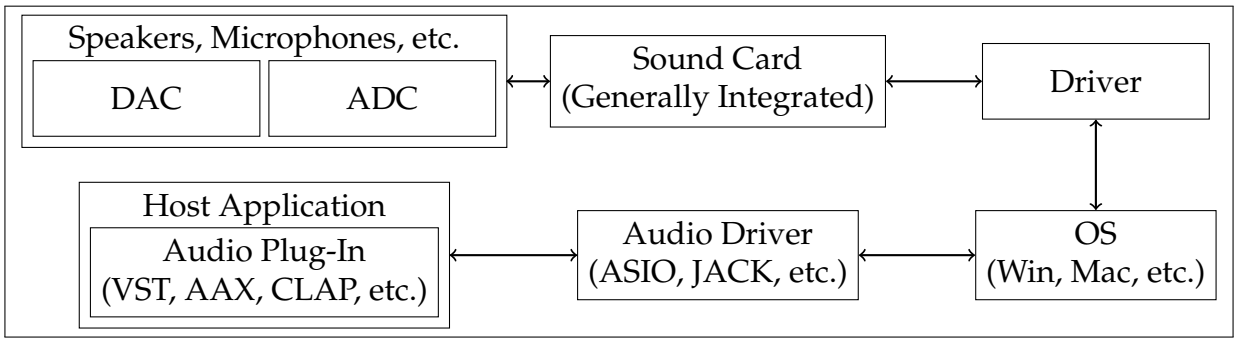


Figure 3.6: The flow of information for audio data in a digital system.

Audio is first processed by the audio plugin under a particular host application. This host application handles several different audio plugins to generate and manipulate sound. This audio is then processed by the audio driver into buffers. Buffers are stored in memory until the processor is interrupted by the audio chipset or sound card, upon which the buffer is sent to be output to the system's speakers. This is completed by the Digital-to-Analog Converter (DAC) which converts the discrete signal into a continuous form for the speaker.

A real-time program does not play audio files akin to one using sound effects for a video game; the program needs to play this sound as it is being generated on-the-fly. As one can imagine, this will provide limitations into the implementation of the program to prevent buffers from being dropped.

3.3 Program Structure & Libraries

The typical structure of an audio program separates the internal processing and the external GUI into multiple threads. The audio thread runs at a speed determined by the host application, often times configurable in its settings. The GUI runs at a slower speed and communicates to the audio thread through messages that can change various parameters upon being recieved. This type of structure is more generally known as a *Model View Controller* design pattern. Specifically, a variation of this model is used that combines the view and controller of the model [21]. Figure 3.7 provides a diagram of this.

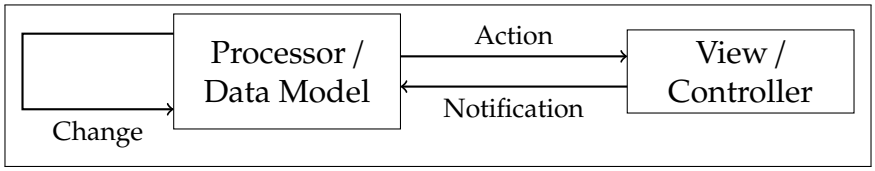


Figure 3.7: A Model of an Audio Plugin Application.

Libraries are available to the programmer that aid the development of audio applications. They do this by providing generators and unit tests to ensure that applications will not have undefined behavior under different host applications. One such library is JUCE, which includes several classes to quickly prototype audio applications [15]. Knowing that the audio thread and GUI must run under different threads, one can intuit that these should be represented as separate classes in code. Figure 3.8 describes a UML diagram of the application; while this one includes additional classes for different features of the program, the *PluginProcessor* and *PluginEditor* classes can be expected for any type of plugin application.

With the design of the plugin taken care of, one can begin implementation of the plugin processor that will manipulate the input signal.

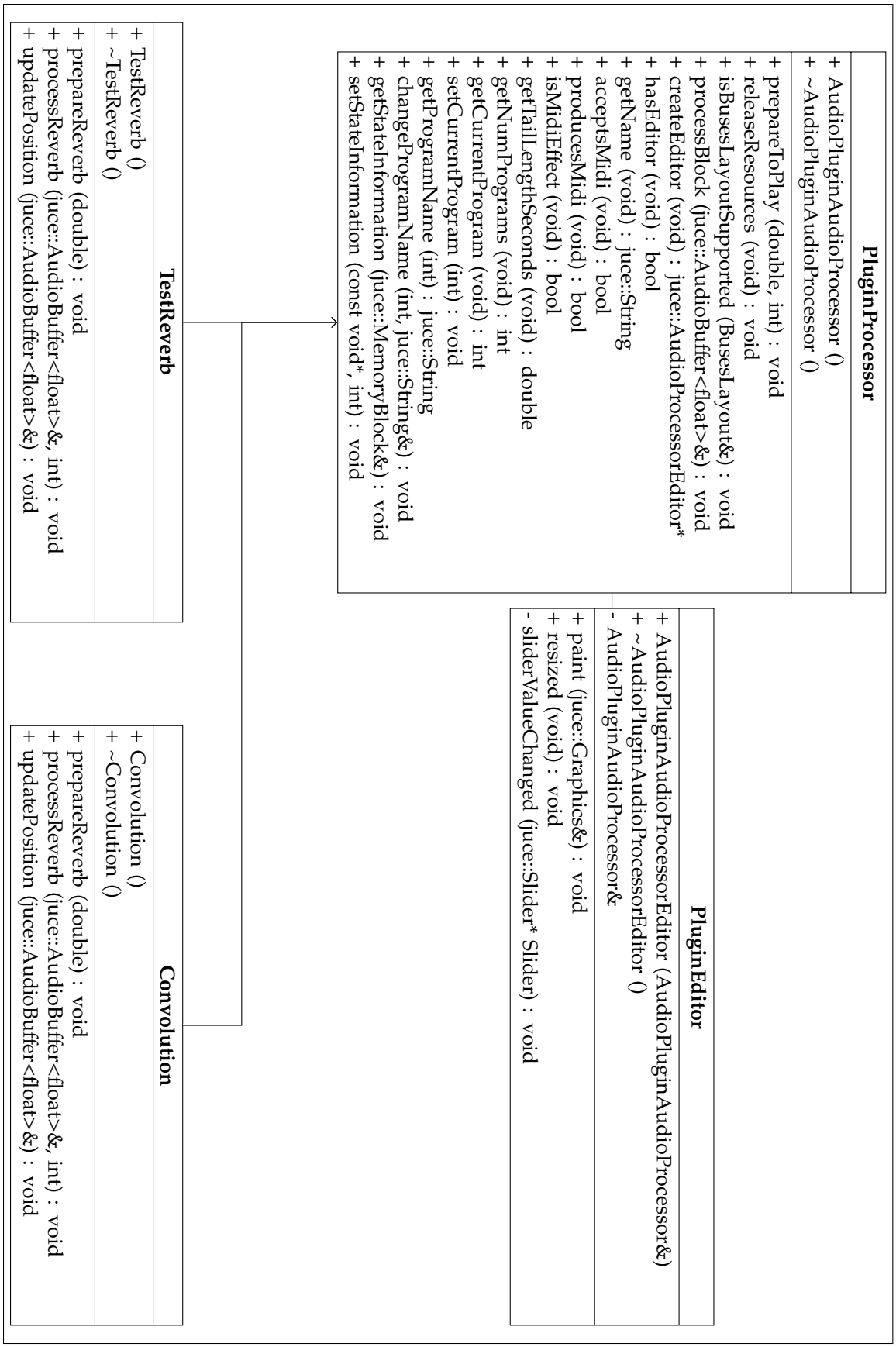


Figure 3.8: UML diagram of the Audio Plugin.

CHAPTER 4

Artificial Reverberation Design

This chapter will begin with a brief overview of digital signal processing to understand artificial reverberation algorithms. After, various methods of reverberation will be explored; specifically going into detail how they work and their corresponding signal flow graphs.

4.1 Filters & Signal Flow Graphs

Thus far, signals have been represented as a function with respect to time. This is not the only manner in which one can visualize audio, however - by computing the *Fourier Transform* of a function, it can be graphed as a function with respect to *frequency*, as opposed to time. At a high level, the Fourier Transform deconstructs a signal to its sinusoids; this was discussed in Chapter 3. However, to compute the Fourier Transform, the previously discussed mathematical representation of sound will need to be reevaluated.

Phase shifted sinusoids are ambiguous as to how they can be represented mathematically. That is, the same sinusoid can be represented as a function of sine *or* cosine. To perform calculations useful to the programmer, one needs a representation of a sinusoidal wave that removes this ambiguity. The *complex sinusoid* does exactly that by encapsulating both functions under the complex plane [23]:

Definition 4.1 Definition of a complex sinusoid:

$$e^{j\omega t} = cos(\omega t) + jsin(\omega t) \tag{4.1}$$

where ω is radians, t is time, and j is $\sqrt{-1}$. With this definition, the Fourier Transform $X(\omega)$ may be calculated from a signal $x(t)$.

Definition 4.2 Definition of the Fourier Transform [31]:

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}dt, \quad \omega \in (-\infty, \infty) \tag{4.2}$$

In a digital setting, this process is computed via the summation of samples from a given input buffer:

Definition 4.3 Definition of the Discrete Fourier Transform [31]:

$$X(\omega_k) = \sum_{n=0}^{N-1} x(t_n)e^{-j\omega_k t_n}, \quad k = 0, 1, 2, ..., N-1 \tag{4.3}$$

Revisiting the Nyquist-Shannon Sampling Theorem, one may graph the bandlimited signal F_{max} and Nyquist Rate F_s with respect to frequency using a DFT. Such an example can be seen below:

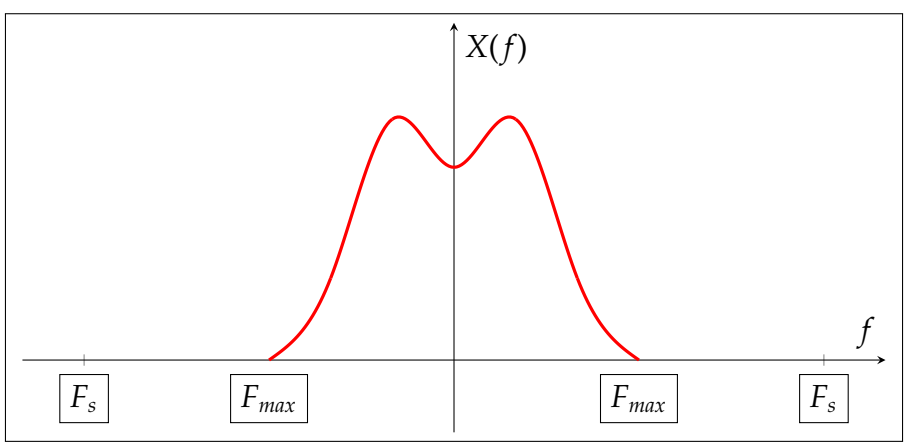


Figure 4.1: An example of a Fourier Transform taken from a bandlimited function.

It is worth reiterating - with this representation, f represents frequency on the x-axis, and $X(f)$ represents the *amplitude* of frequencies. It begs the question - how does this relate to reverberation, exactly?

Recall the way a digital audio workstation functions. Generated sound is sent through various mixer channels to be modified before combining in the master channel. While various effects are available to manipulate sound in the mixer, exactly *how* they work is behind closed doors. Musicians may be familiar with tools such as "lowpass filters" and "highpass filters". Audio engineers expand what exactly a "filter" is to include *all* types of different effects, or black boxes.

With this in mind, a *filter* can be defined as a medium through which a signal enters as an input and exits as an output [30]. It is a type of *black box system* whose inner workings modify the sound in some way; it is inclusive of any type of effect one can think of. By extension, a *digital filter* is a type of filter that operates on digital signals in the form of a computation. This computation can take the form of an equation, or a snippet of code in a computer.

Take the following filter, for example:

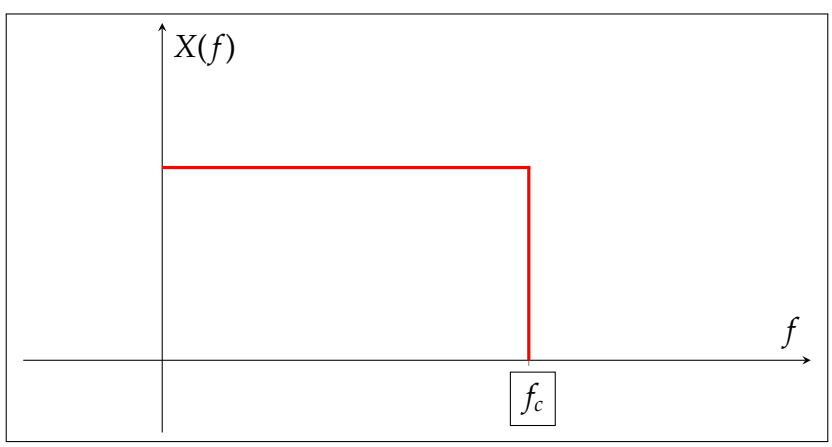


Figure 4.2: The Amplitude Response of a simple lowpass filter.

This type of filter only amplifies the frequencies below a particular *cutoff* f_c. This type of filter is known as a *lowpass filter*, as it only allows frequencies lower than the number

specified to pass through the black box. What $X(f)$ represents in this context is known as the *Amplitude Response* - this is just another way of saying that it is the Frequency Transform of a particular buffer. By definition, the Amplitude Reponse takes the absolute value of $X(f)$ [30].

Understanding filters, one is able to construct graphs consisting of multiple filters. *Signal Flow Graphs* are used to visualize how filters manipulate a provided input and output [30]. They represent a system of linear equations that compute an output signal based on past and present input signals and past output signals. Using signal flow graphs, one can describe a system through which buffers of audio data travel and are processed. The following graph describes one important type of filter known as a *comb filter*:

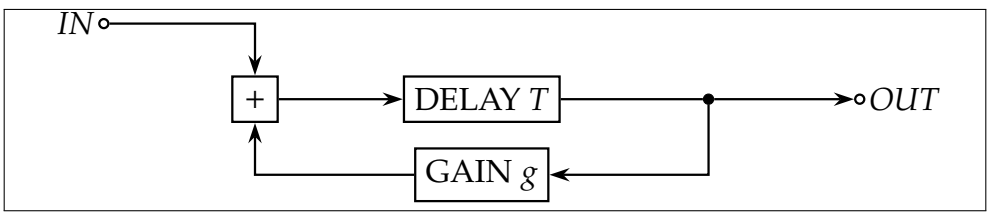


Figure 4.3: The Signal Flow Graph of a Comb Filter [29].

This type of filter describes the process of delaying a signal after some time T [29]. It is arranged in a feedback loop to produce multiple echos and applies a gain g less than one to ensure the signal remains stable. By itself, it is a relatively simple design that producers may know as a "delay effect". In such programs, the time T can often be specified to a specific amount, or be synchronized to the tempo of the song being produced.

A single Comb Filter is not perfect, though. Schroeder derives the Amplitude Response of a Comb Filter to be [28]:

$$|X(\omega)| = \frac{1}{1 + g^2 - 2gcos(\omega(\frac{\pi}{2}))^{1/2}} \tag{4.4}$$

Which results in the following graph, explaining the Comb Filter's name:

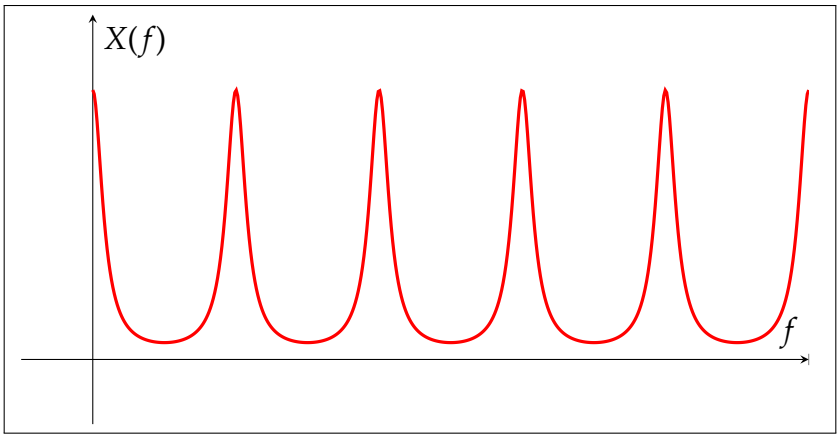


Figure 4.4: The Amplitude Response of a Comb Filter.

These peaks in the Amplitude Response result in what is described as a "metallic" sound, which by itself may not produce a convincing reverb. To achieve a flat frequency response to the comb filter, Schroeder introduces the *Allpass filter* as a method by including an additional feed-forward signal to the output of the comb filter.

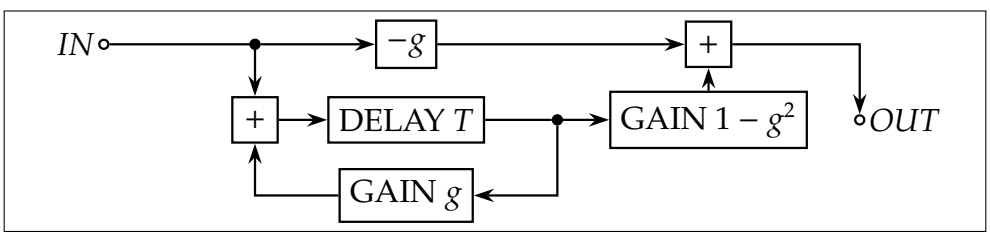


Figure 4.5: The Signal Flow Graph of an Allpass Filter [28].

With the additional feed-forward signal $1 - g^2$, Schroeder derives the Amplitude Response to be [28]:

$$|X(\omega)| = 1 \tag{4.5}$$

which results in a flat response, removing the "metallic" sound of the Comb Filter. With these tools, Schroeder constructs several designs of reverberators that produce sound through purely artificial means. What follows is a description of one such design that he proposes.

4.2 The Schroeder Comb Filter Approach

To achieve a convincing reverb, the *echo density* of the output must be sufficiently large. This measurement is simply the number of echos per second that occur in the resulting output [23]:

$$E_D = \frac{echoes}{second} \tag{4.6}$$

According to Schroeder, the echo density must be greater than 1000 to achieve a "realistic" sound[1]. To accomplish this, one method he provides is by connecting several Allpass Filters in series, which he postulates can be done with five. This provides an echo density "sufficiently close" to the required 1000 [28]. However, Schroeder goes on to introduce a design that includes both Allpass Filters *and* the aforementioned Comb Filters. Despite the imperfect Amplitude Response of the Comb Filter, Schroeder argues that it may represent real rooms better as they likewise contain irregular responses.

To create a sufficient echo density, a number of Comb Filters of varying delay lengths are connected in parallel before the Allpass Filters in series. In his design, he uses four Comb Filters in parallel with two Allpass Filters in series after. The delay lengths of the Comb Filters are said to be incommensurate in order to prevent regular "peaks" from occuring in the output signal [28]. With this design, a sufficient echo density is created to allow for a reverberation time of approximately one second, similar to what one may find in a real acoustic space. Figure 4.6 provides a Signal Flow Graph of this design.

Many artificial reverberation designs use similar principles of repeating the given input signal but can differ in what or how many filters are used. One of the first open-source designs, *Freeverb*[2], used four Allpass Filters in series and eight parallel Filtered-Feedback

[1]The now-accepted value of sufficient echo density is much higher. In 1989, Griesinger proposed that this value should be 10,000 echoes/second *or higher* due to short impulses providing a "grainy" sound, especially for long decay times [11]. Modern reverberation techniques, however, propose adding a "diffuser" step, which removes the graininess that would come from the early reflections of a short impulse. With this additional step, the number of echoes/second can be lowered to 2000-4000 [35].

[2]Interestingly, Freeverb predates a large majority of Open Source Software, as its source code was released publicly on May 24th, 2000 [36].

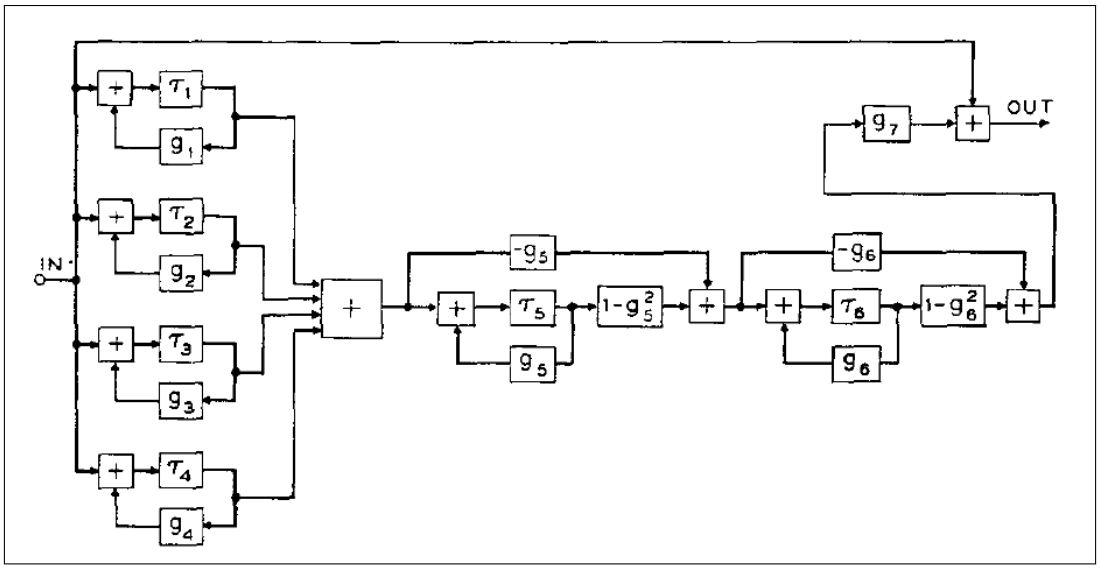


Figure 4.6: A Signal Flow Graph of Schroeder's Reverberator using Comb Filters and Allpass Filters [28].

Comb Filters (or rather, a Comb Filter with a Lowpass Filter placed in the feedback loop) [32]. For each of the designs provided, channels of audio are processed under a single mono channel of audio data. As will be discussed in Chapter 8, splitting audio data into multiple channels is another method of creating a greater echo density.

4.3 Convolution

Convolution is a fundamentally different approach from the Schroeder Comb Filter Approach. Unlike Comb Filters and Allpass Filters, they involve taking an input signal and manipulating it with a particular *impulse response*. In doing so, the two signals are combined (or convolved, rather) such that the output signal contains the product of the input signal and the state of the room measured [23]. While room responses are useful to convolve with, there is nothing stopping one from convolving one signal $x(n)$ with any other arbitrary signal $y(n)$. For discrete signals, the multiplication of Discrete Fourier Transforms (Definition 4.3) correspond to *circular convolution*, which is defined by the following (under the time domain):

Definition 4.4 Definition of Circular Convolution [38]:

$$x(n) \circledast y(n) \xleftrightarrow{DFT} X(k)Y(k) \tag{4.7}$$

$$x(n) \circledast y(n) = \sum_{m=0}^{N-1} x(m)y[(m-n)\%N] \tag{4.8}$$

where $\circledast$ refers to the process of convolving a signal under the time domain, and $X(k)Y(k)$ are the corresponding signals under the frequency domain (similar to that of the Amplitude Responses mentioned earlier). In an intuitive sense, this process can be described as the "sliding" of the impulse response over the input to create overlapping areas [23]. The area under these signals are then computed for each iteration of the slide, resulting in an output signal that is the convolution between the two. Figure 4.7 describes this process visually.

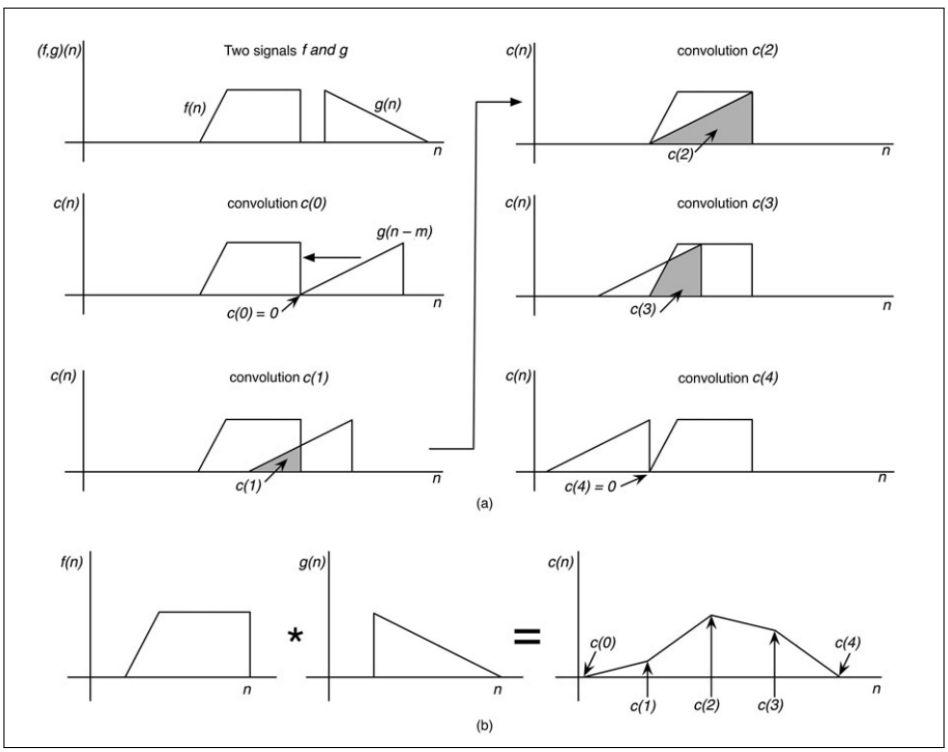


Figure 4.7: A visual representation of two waveforms being convolved [23].

In simpler terms, Convolution is the process of computing the Fourier Transform of a particular input signal and - in the context of artificial reverberation - the impulse response

of a room, multiplying the two signals together such that their frequency responses match what one would hear if the signals were combined (as opposed to played at the same time), and then computing the inverse DFT (summing the output of the DFT back together again). In theory, this process seems quite simple and extremely powerful; in practice, the implementation of the above process is difficult due to the sheer number of computations required [23]. Put simply, the shorter the impulse response, the less computations required to convolve the provided input signal.

CHAPTER 5

Real Acoustic Spaces

This chapter will cover impulse responses and their use in artificial reverberation. It will begin with the mathematical background of rooms and how their echo is measured. Next, several methods of measuring a room's impulse response will be discussed. Finally, the measurements taken from an acoustic space will be provided and the results analyzed.

5.1 Mathematical Background

With the previously mentioned algorithm, it comes naturally that one will need the impulse response of a room in order to have something to convolve with. While several can be found online, measuring the impulse response from a real acoustic space can provide a baseline to evaluate other methods in a unique and novel way. Unlike purely artificial means where delay lines and filters can be tuned to one's taste, convolving the signal with an impulse response should, in theory, result in the same output regardless of the exact algorithm used.

The reverberation of an acoustic space is most commonly measured by its RT_{60} time. This measurement refers to the amount of time it takes for the persisted sound after a short impulse to decay by 60 dB[1]. This is defined as:

[1]It is often difficult to measure a difference of 60 dB, so it is common practice to take a RT_{20} measurement or RT_{30} measurement instead and extrapolate the results to determine the appropriate RT_{60} time [7]

Definition 5.1 Definition of reverberation time:

$$RT_{60} = 0.049\frac{V_R}{A_t} \tag{5.1}$$

$$A_t = S_1\alpha_1 + S_2\alpha_2 + S_3\alpha_3 + \ldots + S_n\alpha_n \tag{5.2}$$

where V_R is the volume of the room, and A_t the total area of absorption in the room, measured in sabins [19]. In this context, sabins refer to the surface area of a wall or floor S_n measured in square feet, multiplied by the absorption coefficient α_n determined by the material of that area. Should the approximate room volume, surface area, and materials be known, the reverberation time can be measured and predicted.

It is at this point that a potential space to measure should be chosen - while any space can be used for reverberation, there are practical limitations that prevent just any room from being measured and recorded. For the purposes of this thesis, Gault Recital Hall - located in Scheide Music Center - was the room chosen for acoustic analysis.

Simplifying the architecture of the space, Gault Recital Hall is largely composed of two areas. The stage itself contains a footprint of approximately $1,473.25ft^2$, made of reflective plywood and plasterboard [12, 24]. The theater space contains a footprint of approximately $2,360ft^2$, with its walls composed of coarse concrete installed in an uneven pattern. As a whole, the total volume - stage and seating included - is approximately $104,609ft^3$, with a surface area of approximately $15,052ft^2$ [12].

To determine the total area of absorption, then, the average absorption coefficient must be found for each surface's material in the hall. Table 5.1 provides the relevant coefficients [33]. As one will observe when measuring the impulse response, coefficients must be averaged across frequencies as the entire range of human hearing is covered. The sum of each surface will result in the value A_t to be used to find the RT_{60} time. In an intuitive sense, a coefficient of 0 will absorb none of the sound - i.e., the surface reflects the sound back into the hall, creating a more reverberant space.

Materials	125 Hz	250 Hz	500 Hz	1 kHz	2 kHz	4 kHz	Average
Cushioned Seats	0.32	0.4	0.42	0.44	0.43	0.48	**0.415**
Wood Parquet	0.04	0.04	0.07	0.06	0.06	0.07	**0.057**
Rough Concrete	0.01	0.02	0.04	0.06	0.08	0.1	**0.052**
Plaster	0.01	0.02	0.02	0.03	0.04	0.05	**0.028**
Metal Panels	0.51	0.78	0.57	0.77	0.9	0.79	**0.72**

Table 5.1: Relevant Absorption Coefficients [33].

Taking each measurement of the hall into consideration[2], the value A_t becomes:

$$A_t = \underbrace{2360.3ft^2 \cdot 0.415}_{\text{Audience Floor}} + \underbrace{1473.75ft^2 \cdot 0.057}_{\text{Stage Floor}} + \underbrace{4953.3ft^2 \cdot 0.052}_{\text{Audience Walls}} + \underbrace{2431ft^2 \cdot 0.028}_{\text{Stage Walls}} + \underbrace{3834.1ft^2 \cdot 0.72}_{\text{Ceiling}}$$

$$A_t = 4149.7 \text{ sabins}$$

This results in a reverberation time of:

$$RT_{60} = 0.049\frac{104,609ft^3}{4149.7} \tag{5.3}$$

$$\therefore RT_{60} = 1.23s \tag{5.4}$$

This begs the question: how is the impulse response measured, exactly?

5.2 Measuring the Impulse Response

To obtain the impulse response, Farina introduces a method that relies on the deconvolution of swept sinusoids [9]. Unlike previous methods, this method does not rely on short impulses (such as popping balloons or firing blanks). Rather, a sinusoid is played through a range of frequencies, which is then deconvolved via its inverse.

[2]These measurements simplify the structure of the room heavily, which assume that the room is composed of two rectanglular prisms - one for the stage and one for the audience. In reality, the architecture of Gault is more complex - most notably containing steps leading to the stage with curved walls along the stage.

To measure the impulse response of Gault Recital Hall, a logarithmic sine sweep of the room was performed, spanning the range of 20 Hz - 20,000 Hz in 10s. These were played from Bag End TA6000-I speakers located approximately halfway up the room on stage left and stage right, with an array of Community IHP-3564 Subwoofers on the ceiling in the middle [16]. Back on the ground, the response of the sweep was recorded with a Zoom XYH-6 X/Y Capsule placed roughly in the center of the room. Several measurements were taken for redundancy, in case any one recording did not suffice.

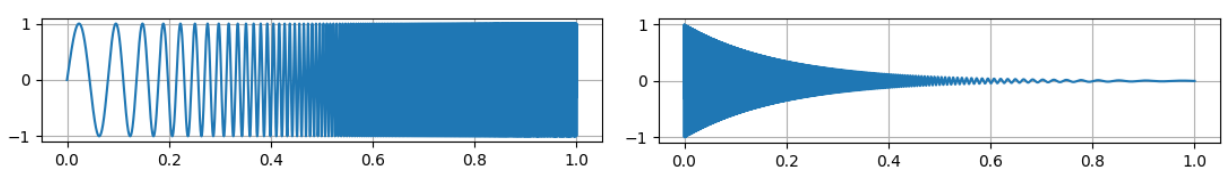
The sine sweep itself was generated via Room EQ Wizard, a program used to measure acoustic spaces [25]. The sweep follows the equation:

$$x(t) = sin\left(\frac{\omega_1 T}{\ln\left(\frac{\omega_2}{\omega_1}\right)} e^{\frac{t}{T}\ln\left(\frac{\omega_2}{\omega_1}\right)} - 1\right)$$

where ω_1 and ω_2 are the starting and ending frequencies, respectively, and T is the duration of the sweep in seconds [9]. After performing the generated sweep, the response was then processed by convolving the raw output audio with the inverse filter $f(t)$ [13]:

$$f(t) = \frac{x_{rev}(t)}{e^{\frac{t}{T}\ln\left(\frac{\omega_2}{\omega_1}\right)}}$$

which is created by utilizing a time reversed $x_{rev}(t)$. The following graphs depict the logarithmic sine sweep and inverse filter, respectively:



(a) Logarithmic Sinusoid Sweep $x(t)$. **(b)** Inverse filter $f(t)$ from $x(t)$.

Figure 5.1: A graph of (a) the Logarithmic Sinusoid Sweep and (b) its inverse.

5.3 Results of a Sinusoid Sweep

In practice, tools are available that calculate the inverse filter based on the impulse response. To accomplish this, raw audio files were processed with ReaVerb - a VST application

included with Reaper. This program allows for impulse response generation by providing the raw audio and the original impulse, automatically generating the inverse filter. Through this program, the following impulse response was generated from the raw audio:

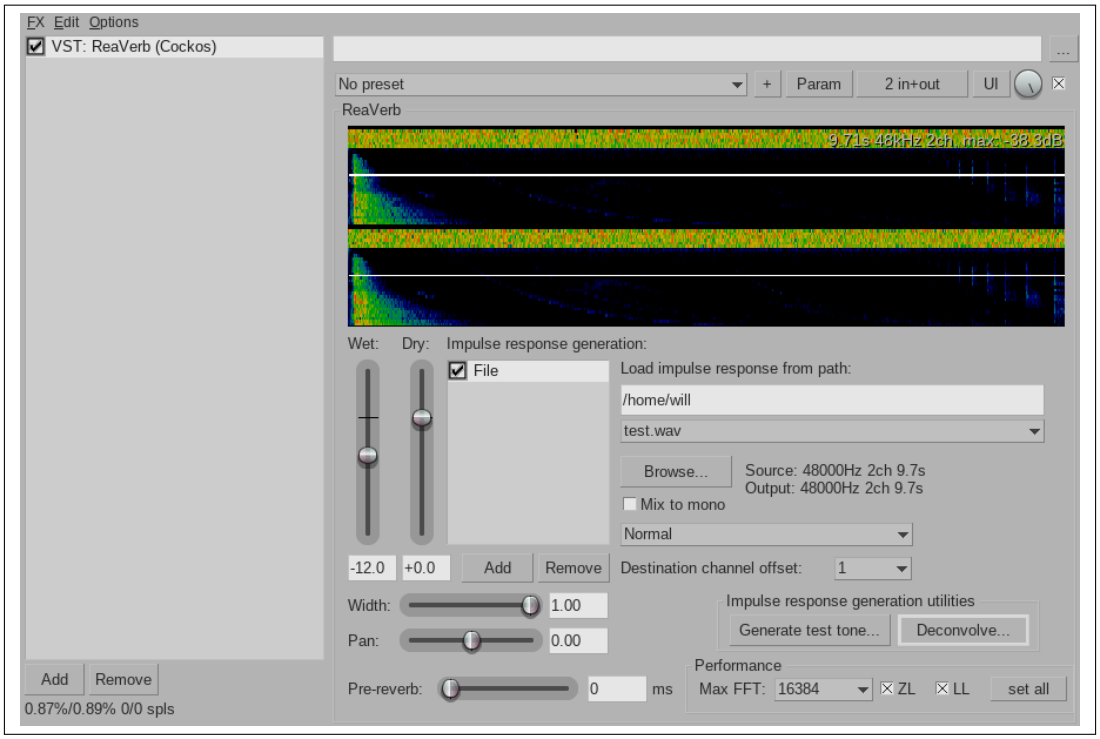


Figure 5.2: A screenshot of the Impulse Response of Gault via Reaverb.

Viewing the IR as a function of time does not reveal much.

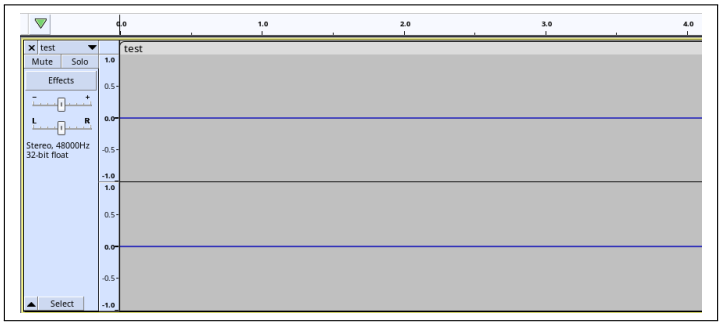


Figure 5.3: A screenshot of the Impulse Response as a function of amplitude and time.

However, by normalizing the audio output, one is able to clearly see the impulse response.

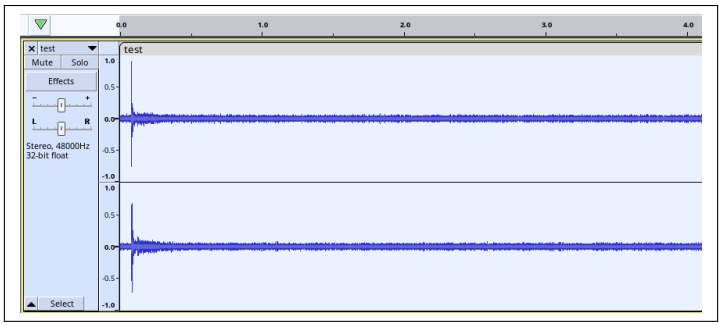


Figure 5.4: The IR normalized.

Some may notice in Figures 5.2 and 5.4 that there appears to be unwanted noise in the IR. By applying a lowpass filter at approximately 17 kHz, this noise is removed.

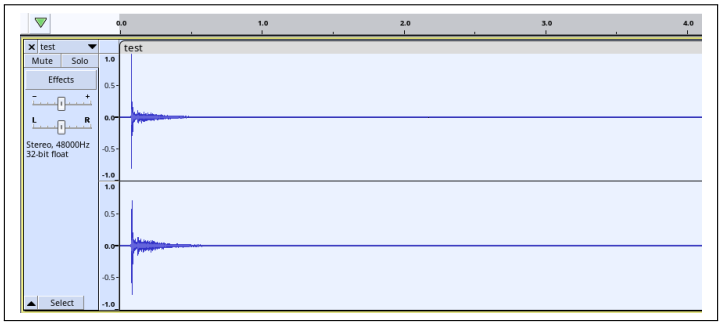
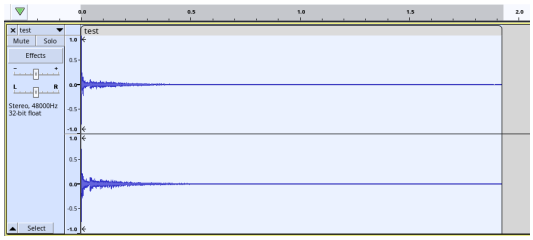
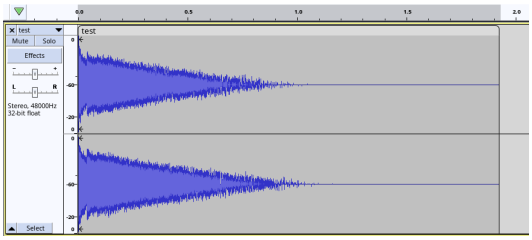


Figure 5.5: The IR normalized and filtered.

Lastly, the audio file is then cropped to remove areas of silence. Viewing the Impulse Response's amplitude with regard to dB, the resulting RT_{60} time is within 15% of the predicted value at approximately 1.05s:



(a) The IR cropped.



(b) The IR with regard to dB.

Figure 5.6: The IR as a function of (a) its amplitude with respect to time and (b) its dB with respect to time.

CHAPTER 6

The Software

To bring everything together, this chapter will cover how the application was created using JUCE. It will begin with a description of the data structures that were used before describing their implementation. Then, the process of creating the user interface will be described. Finally, it will end with how the software runs under several different host applications.

6.1 Data Structures and Implementation

Both Comb Filters and Allpass Filters use a queue ADT for their implementation. However, they rely on an implementation that uses a *circular array* so that audio data can be continuously fed into the application. In other words, an array with modular arithmetic is used so that audio data can be continuously queued. This ignores the usual problem that come from circular arrays; that current lengths can be ambiguous for the same *front* and *back* pointer [2]. For the purposes that this data structure is being used for, all that matters is that data can be continuously processed in the system and overwritten when new data comes in.

To implement a Comb Filter, one can imagine an array of floats representing a buffer of data to be processed. A subsection of this buffer will be specified as the signal to repeat of an arbitrary length. This is the delay buffer. To match the behavior expected of a Comb

Filter in code, one can imagine filling a temporary buffer with data. One can then read this data back to the original buffer once it has been processed (i.e., the write pointer of the circular array has moved past the data read thus far). JUCE provides a datatype `AudioBuffer<Type>` that allows the programmer to copy data from one buffer to the next (or rather, the primary buffer to output and a buffer containing the temporary delayed signal) - however, it is up to the programmer to implement the circular behavior so that undefined results do not occur.

Listing 6.1: The high level code of a Comb Filter.

```
delayLine.fillBuffer (buffer, channel);
delayLine.readFromBuffer (buffer, delayLine, channel);
delayLine.fillBuffer (buffer, channel);
```

This circular behavior can be defined as a custom class `DelayLine`, whose members are defined in Listing A.1. As part of this class, to implement `fillBuffer()`, JUCE provides functions under the `juce_audio_basics/juce_audio_basics.h` header. These include the functions `getNumSamples()`, `copyFrom()`, and `getWritePointer()`. Essentially, the function `fillBuffer()` first checks the size of the delay buffer. Upon doing so, if the write position (the *front* pointer of the circular array) is within the bounds of the delay buffer, then the audio data can simply be copied from the delay buffer to the buffer via the `copyFrom()` function. If not, care is taken to calculate the correct length of the delay buffer and how many samples are present between the *head* and the *back* pointer of the delay line. The `copyFrom()` function can then be used in a similar manner.

This takes care of filling the buffer with the data sent through the delay line (Figure 4.3, filling the buffer with the delay of time T). However, `readFromBuffer()` must read this data to be sent back through the filter as an input with some gain g. To calculate the gain parameter, the naive approach would be to set the variable to some constant. However, this does not work as the length of the decay would be different for each delay time T.

To ensure that each delay line does not have different lengths, g must be calculated as a function with respect to the length of time that the programmer wishes to have the Comb Filter last. This can be done with the following equation:

$$g = 10^{\frac{-3DT_s}{RT_{60}}}$$

where D is the delay line length in seconds, T_s the sample rate, and RT_{60} the reverberation time in seconds [23]. However, one can simplify this further as the sample rate conversion is simply an intermediary to convert seconds to discrete buffers. If working purely with lengths of buffers, this simplifies to:

$$g = 10^{\frac{-3D_b}{RT_b}}$$

where D_b is the delay line length in buffers and RT_b is likewise the RT_{60} time in buffers. This translates to the following code:

Listing 6.2: Code to calculate g within a delay line.

```
auto g = pow(10, ((double) (-3 * (delayTime / rtTime))));
```

It is worth reiterating, the gain g applied during the `readFromBuffer()` step *must* be calculated for each different delay length. For multiple delays, if a constant g is applied for each different delay length, the comb filters will finish decaying at different points in time. This calculation avoids this. The function separates this approach from being just a number of delays played at once - they all contain the same RT_{60} time, despite being of different delay lengths with each repeated signal.

With the gain parameter handled, the `readFromBuffer()` function can be implemented by adding the delayed signal back to the input buffer with the calculated gain applied. This is different from the copy step as the signal is not being replaced - the new signal must be the sum of the next incoming signal with the (now delayed) previous. JUCE contains the function `addFromWithRamp()` that accomplishes this.

Listing 6.3: Code to take the sum of two signals.

```
auto bufferSize = buffer.getNumSamples();
auto delayBufferSize = delayBuffer.getNumSamples();

auto readPosition = writePosition - delayTime;

if (readPosition < 0) {
    readPosition += delayBufferSize;
}

if (readPosition + bufferSize < delayBufferSize) {
    buffer.addFromWithRamp (channel, 0, delayBuffer.
        getReadPointer (channel, readPosition), bufferSize, g, g
        );
}
else {
    auto numSamplesToEnd = delayBufferSize - readPosition;
    buffer.addFromWithRamp (channel, 0, delayBuffer.
        getReadPointer (channel, readPosition), numSamplesToEnd,
         g, g);

    auto numSamplesAtStart = bufferSize - numSamplesToEnd;
    buffer.addFromWithRamp (channel, numSamplesToEnd,
        delayBuffer.getReadPointer (channel, 0),
        numSamplesAtStart, g, g);
}
```

Lastly, it is the responsibility of the programmer to update the *head* pointer to ensure that the circular buffer stays within its intended space in memory and prevents undefined behavior from occuring.

Listing 6.4: Code to implement circular behavior in an array.

```
auto bufferSize = buffer.getNumSamples();
auto delayBufferSize = delayBuffer.getNumSamples();

writePosition += bufferSize;
writePosition %= delayBufferSize;
```

Placing each component in context, these functions work together by being called

within a greater `processBlock()` function that takes a buffer, applies some processing, and returns the buffer back to the DAW. This function is run every time a buffer is sent from the DAW to the VST application. As mentioned in Chapter 3, there is only so much time for processing to be completed, so code must be efficient enough to avoid dropped buffers. The code within `processBlock()` begins by checking how many channels are being processed within the buffer. Most cases, this number is either *mono* (1 channel) or *stereo* (2 channels). Upon determining the correct number of channels, the program then iterates through a *for* loop to process the audio data for each channel. Here, one can begin by making a temporary copy of the buffer to process (allowing the programmer to make a separate *wet* and *dry* mix). This temporary buffer is then sent to a custom class `TestReverb` that processes the buffer through the custom function `processReverb()`. Here, the signal is separated into four separate channels - or, Comb Filters, rather - and processes the audio in the same manner done in Listing 6.1. Each channel of audio is then summed back with the original temporary buffer using JUCE's `addFrom()` function. The programmer can specify any number of Comb Filters in parallel during this step. In a similar manner, Allpass Filters can be implemented by applying a gain of $1 - g^2$ to the signal (post-Comb Filter) that is summed with the inverse of the signal $-g$ prior to the Comb Filter being applied. Upon processing being completed, the function updating the *head* pointer can be called, and the program is ready to recieve the next buffer from the DAW.

Convolution reverbs can be implemented using a similar approach, but can be aided through the use of JUCE's DSP class. This is done by creating a temporary buffer similar to that of the Schroeder Comb Filter approach, but by copying the buffer to a `juce::dsp::AudioBlock<Type>` object, as opposed to a separate buffer. This object automatically handles the buffer in a circular array, taking care of needed to update the *head* pointer as mentioned previously. Upon checking that an Impulse Response is loaded in memory, the program simply calls a `process()` function that automatically handles the FIR filter that is described in Chapter 4.

Listing 6.5: Code to process a buffer via Convolution.

```
if (irLoader.getCurrentIRSize() > 0) {
    irLoader.process(juce::dsp::ProcessContextReplacing<float>(
        block));
}
```

Regardless of the implementation chosen, the programmer must allocate any memory required to allow processing prior to the `processBlock()` function being called. To do this, the `prepareToPlay()` function is called whenever audio data is recieved for the first time by the application (i.e., a buffer that is not silent is sent to the application). It is here that any pre-processing is done within the application, which includes initializing any arrays to be used as delay lines within the application. Private members `channelA` and `combOne` represent some such values, respectively, that describe how the `prepareToPlay()` function might be called within an audio application.

Listing 6.6: Code to initialize values to process.

```
void AudioPluginAudioProcessor::prepareToPlay (double
    sampleRate, int samplesPerBlock) {
    tmp.clear();
    tmp.setSize(2, sampleRate * 2);
    auto delayBufferSize = sampleRate * 2;

    channelA.getBuffer().clear();
    channelA.getBuffer().setSize (2, (int) delayBufferSize);
    combOne.clear();
    combOne.setSize(2, delayBufferSize);

    channelA.setDelay((int) 2423);
}
```

Through these functions, each design can be implemented in code. Should the user wish to manipulate parameters in real-time, however, additional functionality must be implemented.

6.2 User Interface

As previously discussed in Chapter 3, the structure of an audio plugin mandates that the user interface and backend processing are separated. While the code mentioned previously handles the back end processing, it is up to the programmer to implement the user interface so that the user can specify various parameters of the plugin without modifying the code manually. This includes specifying items such as room size, gain, and wet/dry mix.

To do this, private members can be added to the `PluginEditor` class and `PluginProcessor` class respectively. Private members in the `PluginEditor` class handle the specific frontend objects - that is, the sliders, knobs, and menu items themselves. Listing 6.7 describes how one such UI element can be implemented from the class constructor.

Listing 6.7: Code to create a UI element.

```
gain.setSliderStyle (juce::Slider::LinearVertical);
gain.setRange (0.0f, 1.0f, 0.01f);
gain.setTextBoxStyle (juce::Slider::NoTextBox, false, 90, 0);
gain.setPopupDisplayEnabled (true, false, this);
gain.setTextValueSuffix ("Gain");
gain.setValue (0.8f);
```

However, this does not take into account how the two threads communicate with one another. To do this, JUCE contains a private `juce::Slider::Listener` object under the `juce_gui_basics/juce_gui_basics.h` header that allows the program to track the current value of the user interface. Upon doing so, the GUI updates the state of the corresponding `PluginProcessor` value to align with the frontend. Due to each thread running at different speeds, changing parameters in the frontend may introduce a minor amount of error when processing. However, JUCE additionally includes optional functions such as `addFromWithRamp()` that can be used where this may be an issue.

Listing 6.8: Code to update backend parameters.

```
void AudioPluginAudioProcessorEditor::sliderValueChanged (juce
    ::Slider* slider)
{
    processorRef.proc_gain = gain.getValue();
}
```

Through these principles, multiple UI elements can be added that allow the programmer to update any parameter in real time.

6.3 Compatibility with DAWs

Testing the plugin on Debian GNU/Linux 12, it is verified working on DAWs such as Reaper and Renoise. It is expected to work on Windows as well, but may require additional code signing to compile in Mac OSX without enabling developer mode.

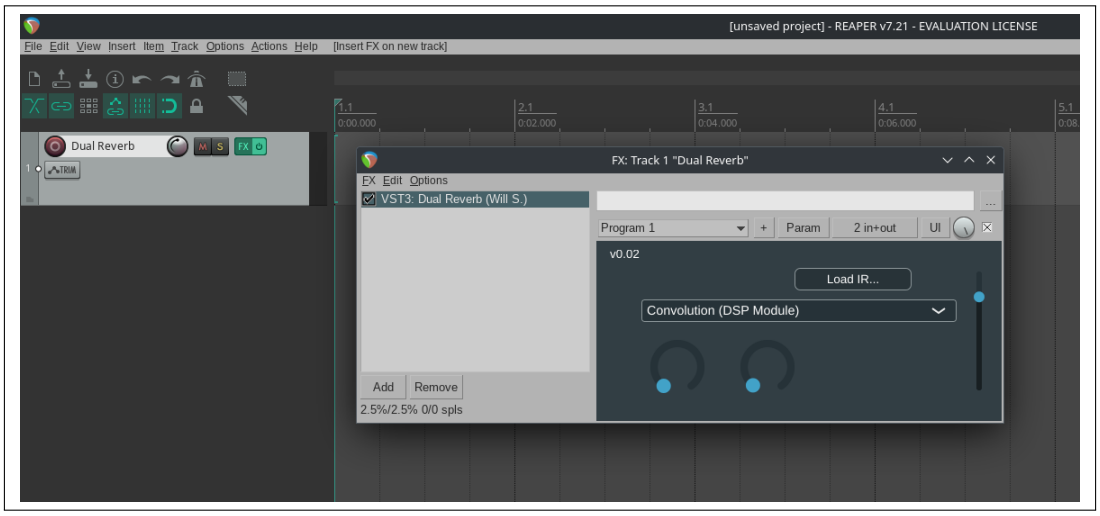


Figure 6.1: The reverb plugin running within Reaper.

CHAPTER 7

Results

This section will discuss how the two different approaches to artificial reverberation perform. It will begin by evaluating how the echo density of the Schroeder Comb Filter approach increases with the number of filters provided. Next, a comparison will be made between the Schroeder Comb Filter approach and the Convolution approach to determine how the purely artificial method compares to a reverb created from the Impulse Response of a real acoustic space. Finally, the chapter will end with how one can improve upon these designs and some of the drawbacks between the two approaches.

7.1 Evaluating the Echo Density

Each additional Comb Filter added to the reverberator in parallel increases the echo density linearly. With the aforementioned four Comb Filters, the echo density is 100. This figure can be increased with an implemented allpass filter in series within the reverberator. In doing so, the resulting echo density can be increased to the required 1000.

With the short impulse provided as an input, the waveform visibly shows where each delay line repeats within the reverberator. While this can create an unwanted effect for lower echo densities, this problem is mitigated with instruments with a longer attack, such as a piano, or violin.

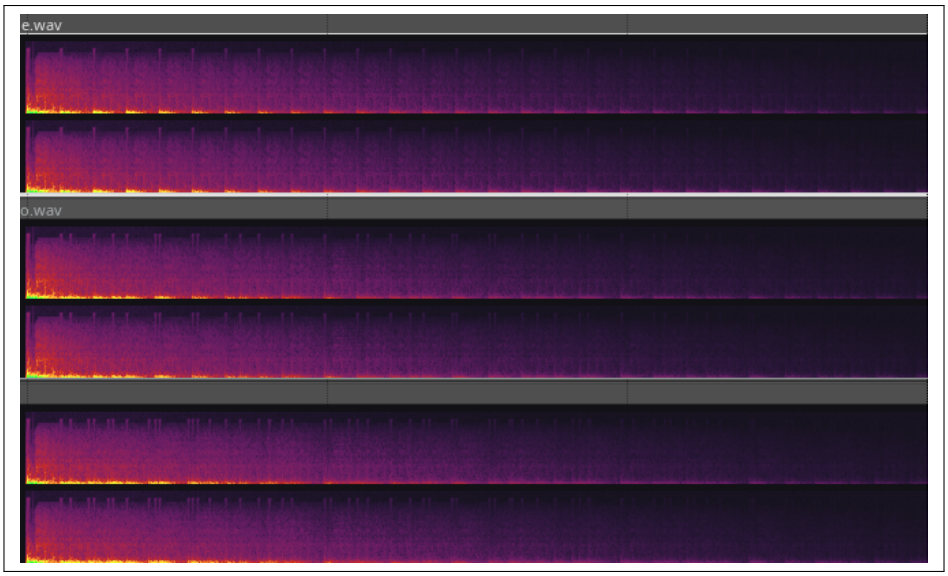


Figure 7.1: Several waveforms with a number of Comb Filters in parallel.

7.2 Comparisons With a Real Acoustic Space

Comparing this with the Convolution approach, one can ascertain that the Impulse Response provides a more realistic sound, but JUCE's DSP library prevents this from truly taking advantage of this in its current state. Once again, short impulse responses create regular peaks that can allow one to ascertain where exactly the delay line is repeating the given signal. However, this is likely due to the high performant requirements of this type of application, as the number of computations required increases exponentially the longer the Impulse Response provided is. While the current implementation is limited by the high performant requirements of the application, other reverberators using the same IR do provide highly convincing and realistic results, such as from ReaVerb.

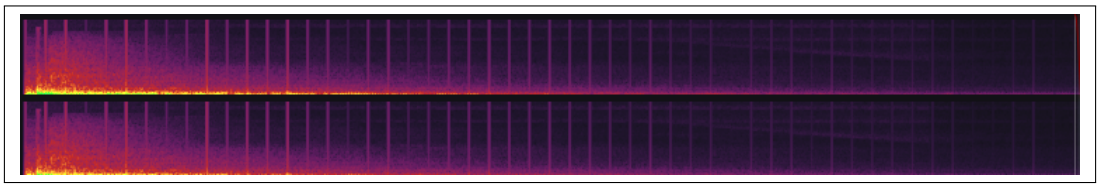


Figure 7.2: A waveform of a reverberated snare drum from a Convolved IR.

CHAPTER 8

Conclusion and Future Work

This barely scratches the surface. While the topics mentioned in this thesis cover the fundamentals of artificial reverberation, there are many different designs and improvements that can be made over the program as it is described here. Schroeder reverberators are considered a specific case to the more generalized structure known as a *Feedback Delay Network (FDN)* [27]. A modernized artificial reverberator will use this type of structure with a diffuser step, should one take the route of a purely artificial design. Through this design, a reverberator would split the audio into several channels - similar to that of Comb Filters in parallel - but would manipulate the signal by distributing the amplitude amongst the matrix of channels to provide a more diffuse sound [20]. This is just one area that is possible to improve upon.

Several improvements can be made during the prediction and measurement of Impulse Responses, as well. The Absorption Coefficients found in Table 5.1 were chosen as what best approximated the materials found in the room - a better selection of coefficients would likely yield a more accurate result. Along with this, the simplification of the room geomety would introduce some amount of error in the prediction as well - a better prediction would take each surface into account, including steps and curves of the performance walls. When recording, better practice would be to include a marker to better align the raw audio files with the impulse to deconvolve with - this is likely the culprit for the noise found in Figures 5.2 and 5.4.

The code itself can be improved upon, as well. In its current state, the Convolution approach includes several glitches in the audio output, which is likely due to the number of calculations required for the length of the Impulse Response. While building a "Release" build of the application *significantly* improves the latency of this approach, it still can be improved upon with better tuning and usage of JUCE's DSP class. Similarly, the code behind the Schroeder Comb Filter approach can be improved through less copying of buffers - there is likely a more efficient solution that involves less usage of JUCE's classes that would allow for a larger number of filters in parallel before performance takes a hit, similar to what Freeverb was able to accomplish.

Despite this, this thesis provides a groundwork for any future audio engineer wishing to develop this type of application. I encourage anyone wishing to learn more to search and read through the citations used, as they contain a wealth of knowledge that this thesis only uses a fraction of. Anyone wishing to view the code can find it in Appendix A and online [8]. While the topics and filters used are specifically in the context of artificial reverberation, these same techniques can be applied to a wide variety of other effects, including distortion, vocoding, and autotune.

APPENDIX A

Code Classes

Listing A.1: DelayLine.h

```
#pragma once

#include <juce_audio_processors/juce_audio_processors.h>

class DelayLine
{
public:
    DelayLine();
    ~DelayLine();

    void fillBuffer (juce::AudioBuffer<float>& buffer, int
        channel);
    void readFromBuffer (juce::AudioBuffer<float>& buffer, juce
        ::AudioBuffer<float>& delayBuffer, int channel);
    void updateBufferPosition (juce::AudioBuffer<float>& buffer
        , juce::AudioBuffer<float>& delayBuffer);
    void setDelay (int time);
    void setSize (int size);

    juce::AudioBuffer<float>& getBuffer ();

private:
    // For circular array data structure
    juce::AudioBuffer<float> delayBuffer;
    int writePosition { 0 };
    int delayTime;
    int roomSize;
};
```

Listing A.2: DelayLine.cpp

```
#include "DelayLine.h"

DelayLine::DelayLine ()
{
}

DelayLine::~DelayLine ()
{
}

void DelayLine::fillBuffer (juce::AudioBuffer<float>& buffer, int channel) {
    auto bufferSize = buffer.getNumSamples();
    auto delayBufferSize = delayBuffer.getNumSamples();

    if (delayBufferSize > bufferSize + writePosition)
    {
        delayBuffer.copyFrom (channel, writePosition, buffer.getWritePointer(channel), bufferSize);
    }
    else
    {
        auto numSamplesToEnd = delayBufferSize - writePosition;
        delayBuffer.copyFrom (channel, writePosition, buffer.getWritePointer(channel), numSamplesToEnd);

        auto numSamplesAtStart = bufferSize - numSamplesToEnd;
        delayBuffer.copyFrom (channel, 0, buffer.getWritePointer(channel, numSamplesToEnd), numSamplesAtStart);
    }
}

void DelayLine::readFromBuffer (juce::AudioBuffer<float>& buffer, juce::AudioBuffer<float>& delayBuffer, int channel)
{
    auto bufferSize = buffer.getNumSamples();
    auto delayBufferSize = delayBuffer.getNumSamples();

    auto readPosition = writePosition - delayTime;
    if (readPosition < 0) {
        readPosition += delayBufferSize;
    }

    auto g = pow(10, ((double)(-3 * ((double) delayTime) / (int) 50400)));
```

```
    if (readPosition + bufferSize < delayBufferSize) {
        buffer.addFromWithRamp (channel, 0, delayBuffer.
            getReadPointer (channel, readPosition), bufferSize,
            g, g);
    }
    else {
        auto numSamplesToEnd = delayBufferSize - readPosition;
        buffer.addFromWithRamp (channel, 0, delayBuffer.
            getReadPointer (channel, readPosition),
            numSamplesToEnd, g, g);

        auto numSamplesAtStart = bufferSize - numSamplesToEnd;
        buffer.addFromWithRamp (channel, numSamplesToEnd,
            delayBuffer.getReadPointer (channel, 0),
            numSamplesAtStart, g, g);
    }
}

void DelayLine::updateBufferPosition (juce::AudioBuffer<float>&
    buffer, juce::AudioBuffer<float>& delayBuffer)
{
    auto bufferSize = buffer.getNumSamples();
    auto delayBufferSize = delayBuffer.getNumSamples();

    writePosition += bufferSize;
    writePosition %= delayBufferSize;
}

void DelayLine::setDelay (int time)
{
    delayTime = time;
}

void DelayLine::setSize (int size)
{
    roomSize = size;
}

juce::AudioBuffer<float>& DelayLine::getBuffer ()
{
    return delayBuffer;
}
```

Listing A.3: TestReverb.h

```
#pragma once
#include <juce_audio_processors/juce_audio_processors.h>
#include "components/DelayLine.h"
#include "juce_audio_basics/juce_audio_basics.h"

class TestReverb
{
public:
    TestReverb();
    ~TestReverb();

    void prepareReverb(double sampleRate);
    void processReverb(juce::AudioBuffer<float>& buffer, int
        channel);
    void updatePosition(juce::AudioBuffer<float>& buffer);

private:

    // Comb filters in parallel
    juce::AudioBuffer<float> combOne;
    juce::AudioBuffer<float> combTwo;
    juce::AudioBuffer<float> combThr;
    juce::AudioBuffer<float> combFou;

    // DelayLine for each comb filter
    DelayLine channelA;
    DelayLine channelB;
    DelayLine channelC;
    DelayLine channelD;

};
```

Listing A.4: TestReverb.cpp

```
#include "TestReverb.h"
#include "juce_audio_basics/juce_audio_basics.h"
#include "juce_core/juce_core.h"

TestReverb::TestReverb()
{
}

TestReverb::~TestReverb()
{
```

```
}

void TestReverb::prepareReverb(double sampleRate)
{
    // preparing delay line stuff
    auto delayBufferSize = sampleRate * 2;

    channelA.getBuffer().clear();
    channelA.getBuffer().setSize (2, (int) delayBufferSize);

    channelB.getBuffer().clear();
    channelB.getBuffer().setSize (2, (int) delayBufferSize);

    channelC.getBuffer().clear();
    channelC.getBuffer().setSize (2, (int) delayBufferSize);

    channelD.getBuffer().clear();
    channelD.getBuffer().setSize (2, (int) delayBufferSize);

    combOne.clear();
    combOne.setSize(2, delayBufferSize);

    combTwo.clear();
    combTwo.setSize(2, delayBufferSize);

    combThr.clear();
    combThr.setSize(2, delayBufferSize);

    combFou.clear();
    combFou.setSize(2, delayBufferSize);

    channelA.setDelay((int) 2423);
    channelB.setDelay((int) 3121);
    channelC.setDelay((int) 4451);
    channelD.setDelay((int) 5903);
}

void TestReverb::processReverb(juce::AudioBuffer<float>& buffer
    , int channel)
{
    // COMB FILTERS
    // Start by holding the current dry signal as a temp var
    combOne.makeCopyOf(buffer, 1);
    combTwo.makeCopyOf(buffer, 1);
    combThr.makeCopyOf(buffer, 1);
    combFou.makeCopyOf(buffer, 1);

```

```
    // Complete each feedback loop and add them to their respective comb filter
    channelA.fillBuffer (combOne, channel);
    channelA.readFromBuffer (combOne, channelA.getBuffer(), channel);
    channelA.fillBuffer (combOne, channel);

    // Once fed back, add each comb filter to the wet signal
    buffer.addFrom(channel, 0, combOne, channel, 0, combOne.getNumSamples(), -1.0f);

    channelB.fillBuffer (combTwo, channel);
    channelB.readFromBuffer (combTwo, channelB.getBuffer(), channel);
    channelB.fillBuffer (combTwo, channel);

    buffer.addFrom(channel, 0, combTwo, channel, 0, combTwo.getNumSamples(), -1.0f);

    channelC.fillBuffer (combThr, channel);
    channelC.readFromBuffer (combThr, channelC.getBuffer(), channel);
    channelC.fillBuffer (combThr, channel);

    buffer.addFrom(channel, 0, combThr, channel, 0, combThr.getNumSamples(), -1.0f);

    channelD.fillBuffer (combFou, channel);
    channelD.readFromBuffer (combFou, channelD.getBuffer(), channel);
    channelD.fillBuffer (combFou, channel);

    buffer.addFrom(channel, 0, combFou, channel, 0, combFou.getNumSamples(), -1.0f);

}

void TestReverb::updatePosition(juce::AudioBuffer<float>& buffer)
{
    channelA.updateBufferPosition (buffer, channelA.getBuffer());
    channelB.updateBufferPosition (buffer, channelB.getBuffer());
    channelC.updateBufferPosition (buffer, channelC.getBuffer);
```

```
    channelD.updateBufferPosition (buffer, channelD.getBuffer()
        );
}
```

Listing A.5: PluginProcessor.h

```
#pragma once

#include <juce_audio_processors/juce_audio_processors.h>
#include <juce_dsp/juce_dsp.h>
#include "TestReverb.h"
#include "juce_audio_basics/juce_audio_basics.h"

class AudioPluginAudioProcessor final : public juce::
    AudioProcessor
{
public:

    AudioPluginAudioProcessor();
    ~AudioPluginAudioProcessor() override;


    void prepareToPlay (double sampleRate, int samplesPerBlock)
        override;
    void releaseResources() override;

    bool isBusesLayoutSupported (const BusesLayout& layouts)
        const override;

    void processBlock (juce::AudioBuffer<float>&, juce::
        MidiBuffer&) override;
    using AudioProcessor::processBlock;


    juce::AudioProcessorEditor* createEditor() override;
    bool hasEditor() const override;


    const juce::String getName() const override;

    bool acceptsMidi() const override;
    bool producesMidi() const override;
    bool isMidiEffect() const override;
    double getTailLengthSeconds() const override;

    int getNumPrograms() override;
```

```
    int getCurrentProgram() override;
    void setCurrentProgram (int index) override;
    const juce::String getProgramName (int index) override;
    void changeProgramName (int index, const juce::String&
        newName) override;

    void getStateInformation (juce::MemoryBlock& destData)
        override;
    void setStateInformation (const void* data, int sizeInBytes
        ) override;

    // parameters here
    float proc_gain;
    int proc_roomsize;
    int proc_reverb_type = 1;

    // files for IR
    juce::dsp::Convolution irLoader {juce::dsp::Convolution::
        NonUniform{ 1024 }};
    juce::File savedFile;


private:
    // Reverb classes
    TestReverb reverb;

    // For Convolution reverb
    juce::dsp::ProcessSpec spec;

    // Temporary processing buffer
    juce::AudioBuffer<float> tmp;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (
        AudioPluginAudioProcessor)
};
```

Listing A.6: PluginProcessor.cpp

```
#include "PluginProcessor.h"
#include "PluginEditor.h"
#include "TestReverb.h"

AudioPluginAudioProcessor::AudioPluginAudioProcessor()
     : AudioProcessor (BusesProperties()
                      #if ! JucePlugin_IsMidiEffect
                       #if ! JucePlugin_IsSynth
```

```
                        .withInput  ("Input",  juce::
                            AudioChannelSet::stereo(), true)
                      #endif
                        .withOutput ("Output", juce::
                            AudioChannelSet::stereo(), true)
                    #endif
                        )
{
}

AudioPluginAudioProcessor::~AudioPluginAudioProcessor()
{
}

const juce::String AudioPluginAudioProcessor::getName() const
{
    return JucePlugin_Name;
}

bool AudioPluginAudioProcessor::acceptsMidi() const
{
   #if JucePlugin_WantsMidiInput
    return true;
   #else
    return false;
   #endif
}

bool AudioPluginAudioProcessor::producesMidi() const
{
   #if JucePlugin_ProducesMidiOutput
    return true;
   #else
    return false;
   #endif
}

bool AudioPluginAudioProcessor::isMidiEffect() const
{
   #if JucePlugin_IsMidiEffect
    return true;
   #else
    return false;
   #endif
}

double AudioPluginAudioProcessor::getTailLengthSeconds() const
```

```
{
    return 0.0;
}

int AudioPluginAudioProcessor::getNumPrograms()
{
    return 1;   // NB: some hosts don't cope very well if you tell them there are 0 programs,
                // so this should be at least 1, even if you're not really implementing programs.
}

int AudioPluginAudioProcessor::getCurrentProgram()
{
    return 0;
}

void AudioPluginAudioProcessor::setCurrentProgram (int index)
{
    juce::ignoreUnused (index);
}

const juce::String AudioPluginAudioProcessor::getProgramName (int index)
{
    juce::ignoreUnused (index);
    return {};
}

void AudioPluginAudioProcessor::changeProgramName (int index, const juce::String& newName)
{
    juce::ignoreUnused (index, newName);
}

void AudioPluginAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
{
    // Use this method as the place to do any pre-playback
    // initialisation that you need..
    tmp.clear();
    tmp.setSize(2, sampleRate * 2);

    spec.maximumBlockSize = samplesPerBlock;
    spec.sampleRate = sampleRate;
    spec.numChannels = getTotalNumOutputChannels();

```

```
    irLoader.reset();
    // Prepare reverb if option is selected
    if (proc_reverb_type == 1) {
        // First, prepare the temp buffer that will store the signal to be processed

        // Then call custom reverb function
        reverb.prepareReverb(sampleRate);

    } else if (proc_reverb_type == 2) {

        // else, prepare JUCE DSP Module
        irLoader.prepare(spec);
    }

    // Next, prepare the reverb that will do the processing

    juce::ignoreUnused (sampleRate, samplesPerBlock);
}

void AudioPluginAudioProcessor::releaseResources()
{
    // When playback stops, you can use this as an opportunity to free up any
    // spare memory, etc.
}

bool AudioPluginAudioProcessor::isBusesLayoutSupported (const BusesLayout& layouts) const
{
  #if JucePlugin_IsMidiEffect
    juce::ignoreUnused (layouts);
    return true;
  #else
    // This is the place where you check if the layout is supported.
    // In this template code we only support mono or stereo.
    // Some plugin hosts, such as certain GarageBand versions, will only
    // load plugins that support stereo bus layouts.
    if (layouts.getMainOutputChannelSet() != juce::AudioChannelSet::mono()
     && layouts.getMainOutputChannelSet() != juce::AudioChannelSet::stereo())
        return false;

```

```
    // This checks if the input layout matches the output layout
   #if ! JucePlugin_IsSynth
    if (layouts.getMainOutputChannelSet() != layouts.getMainInputChannelSet())
        return false;
   #endif

    return true;
  #endif
}

void AudioPluginAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer,
                                              juce::MidiBuffer& midiMessages)
{
    juce::ignoreUnused (midiMessages);

    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels  = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i) {
        buffer.clear (i, 0, buffer.getNumSamples());
    }

    // Schroeder Reverberator
    if (proc_reverb_type == 1) {
        for (int channel = 0; channel < totalNumInputChannels; ++channel) {
            // Make copy of dry signal to process
            tmp.makeCopyOf(buffer, 1);

            // Delay
            reverb.processReverb(tmp, channel);

            // Copy wet signal back to dry signal (allpass-kinda)
            buffer.applyGain(0.3f);

            // Apply gain from parameter
            buffer.applyGain(proc_gain);

            buffer.addFrom(channel, 0, tmp, channel, 0, tmp.getNumSamples(), 0.3f);
```

```
        }
        reverb.updatePosition(tmp);
    }
    // tmp for now, change to switch statement?
    if (proc_reverb_type == 2) {
        for (int channel = 0; channel < totalNumInputChannels;
            ++channel) {

            juce::dsp::AudioBlock<float> block {buffer};

            // only process if IR is loaded from GUI
            if (irLoader.getCurrentIRSize() > 0) {
                irLoader.process(juce::dsp::
                    ProcessContextReplacing<float>(block));
            }
        }
    }

}

bool AudioPluginAudioProcessor::hasEditor() const
{
    return true; // (change this to false if you choose to not
        supply an editor)
}

juce::AudioProcessorEditor* AudioPluginAudioProcessor::
    createEditor()
{
    return new AudioPluginAudioProcessorEditor (*this);
}

void AudioPluginAudioProcessor::getStateInformation (juce::
    MemoryBlock& destData)
{
    // You should use this method to store your parameters in
        the memory block.
    // You could do that either as raw data, or use the XML or
        ValueTree classes
    // as intermediaries to make it easy to save and load
        complex data.
    juce::ignoreUnused (destData);

    juce::MemoryOutputStream (destData, true).writeFloat (
        proc_gain);
    juce::MemoryOutputStream (destData, true).writeFloat (
        proc_roomsize);
```

```
    juce::MemoryOutputStream (destData, true).writeFloat (proc_reverb_type);
}

void AudioPluginAudioProcessor::setStateInformation (const void * data, int sizeInBytes)
{
    // You should use this method to restore your parameters from this memory block,
    // whose contents will have been created by the getStateInformation() call.
    juce::ignoreUnused (data, sizeInBytes);

    proc_gain = juce::MemoryInputStream (data, static_cast<size_t> (sizeInBytes), false).readFloat();
    proc_roomsize = juce::MemoryInputStream (data, static_cast<size_t> (sizeInBytes), false).readFloat();
    proc_reverb_type = juce::MemoryInputStream (data, static_cast<size_t> (sizeInBytes), false).readFloat();
}


// This creates new instances of the plugin..
juce::AudioProcessor* JUCE_CALLTYPE createPluginFilter()
{
    return new AudioPluginAudioProcessor();
}
```

Listing A.7: PluginEditor.h

```
#pragma once

#include "PluginProcessor.h"
#include "juce_audio_formats/juce_audio_formats.h"
#include "juce_core/juce_core.h"
#include "juce_gui_basics/juce_gui_basics.h"

class AudioPluginAudioProcessorEditor final : public juce::AudioProcessorEditor,
                                              private juce::Slider::Listener
{
public:
    explicit AudioPluginAudioProcessorEditor (AudioPluginAudioProcessor&);
```

```
    ~AudioPluginAudioProcessorEditor() override;

    void paint (juce::Graphics&) override;
    void resized() override;

private:
    void sliderValueChanged (juce::Slider* slider) override;
    void styleMenuChanged ();
    void openSample ();

    // This reference is provided as a quick way for your editor to
    // access the processor object that created it.
    AudioPluginAudioProcessor& processorRef;

    // Formatmanager keeps track of opened files in the audio plugin
    juce::AudioFormatManager formatManager;
    std::unique_ptr<juce::FileChooser> chooser;

    // pathMutex is used as a background process to avoid problems with unsynchronized loading of IRs (see juce::dsp::Convolution ref)
    juce::CriticalSection pathMutex;
    juce::String chosenPath;

    // Frontend parameters
    juce::Slider roomSize;
    juce::Slider mix;
    juce::Slider gain;
    juce::Label textLabel { {}, "v0.02" };
    juce::ComboBox styleMenu;
    juce::TextButton loadSample;

    JUCE_DECLARE_NON_COPYABLE_WITH_LEAK_DETECTOR (AudioPluginAudioProcessorEditor)
};
```

Listing A.8: PluginEditor.cpp

```
#include "PluginProcessor.h"
#include "PluginEditor.h"

AudioPluginAudioProcessorEditor::AudioPluginAudioProcessorEditor (AudioPluginAudioProcessor& p)
```

```
    : AudioProcessorEditor (&p), processorRef (p)
{
    juce::ignoreUnused (processorRef);

    // UI EDITING!
    roomSize.setSliderStyle (juce::Slider::RotaryVerticalDrag);
    roomSize.setRange (19350, 1935000, 1.0);
    roomSize.setTextBoxStyle (juce::Slider::NoTextBox, false,
        90, 0);
    roomSize.setPopupDisplayEnabled (true, false, this);
    roomSize.setTextValueSuffix (" Room Size");
    roomSize.setValue (1.0);

    mix.setSliderStyle (juce::Slider::RotaryVerticalDrag);
    mix.setRange (0.0, 127.0, 1.0);
    mix.setTextBoxStyle (juce::Slider::NoTextBox, false, 90, 0)
        ;
    mix.setPopupDisplayEnabled (true, false, this);
    mix.setTextValueSuffix (" Wet/Dry");
    mix.setValue (1.0);

    gain.setSliderStyle (juce::Slider::LinearVertical);
    gain.setRange (0.0f, 1.0f, 0.01f);
    gain.setTextBoxStyle (juce::Slider::NoTextBox, false, 90,
        0);
    gain.setPopupDisplayEnabled (true, false, this);
    gain.setTextValueSuffix (" Gain");
    gain.setValue (0.8f);

    loadSample.setButtonText ("Load IR...");

    // Stylemenu options for selecting reverb type
    styleMenu.addItem("Comb + Allpass", 1);
    styleMenu.addItem("Convolution (DSP Module)", 2);

    // Make UI Visible
    addAndMakeVisible (&roomSize);
    addAndMakeVisible (&mix);
    addAndMakeVisible (&gain);
    addAndMakeVisible (&textLabel);
    addAndMakeVisible (&styleMenu);
    addAndMakeVisible (&loadSample);

    setSize (450, 200);

    // ADD LISTENERS FOR UI
    gain.addListener (this);
```

```
    roomSize.addListener (this);

    styleMenu.onChange = [this] {
        styleMenuChanged();
    };
    styleMenu.setSelectedId (1);

    // Load File from Button try #1
    loadSample.onClick = [this] {
        if (styleMenu.getSelectedId() == 2) {
            openSample();
        }
    };

    // boilerplate
    formatManager.registerBasicFormats();

}

AudioPluginAudioProcessorEditor::~
    AudioPluginAudioProcessorEditor()
{
}


void AudioPluginAudioProcessorEditor::paint (juce::Graphics& g)
{
    g.fillAll (getLookAndFeel().findColour (juce::
        ResizableWindow::backgroundColourId));

    g.setColour (juce::Colours::white);
    g.setFont (15.0f);
}

void AudioPluginAudioProcessorEditor::resized()
{
    // This is generally where you'll want to lay out the
        positions of any
    // subcomponents in your editor..

    auto area = getLocalBounds();

    roomSize.setBounds (50, 100, 80, 80);
    mix.setBounds (150, 100, 80, 80);
    gain.setBounds (400, 25, 50, 150);
    textLabel.setBounds (10, 10, 300, 10);
    styleMenu.setBounds (50, 65, 350, 25);
```

```
    loadSample.setBounds (220, 30, 130, 25);
}

void AudioPluginAudioProcessorEditor::sliderValueChanged (juce
    ::Slider* slider)
{
    processorRef.proc_gain = gain.getValue();
    processorRef.proc_roomsize = roomSize.getValue();
}

void AudioPluginAudioProcessorEditor::styleMenuChanged ()
{
    processorRef.proc_reverb_type = (int) styleMenu.
        getSelectedId();
}

void AudioPluginAudioProcessorEditor::openSample () {

    chooser = std::make_unique<juce::FileChooser>("Choose an
        Impulse Response", juce::File::getSpecialLocation(juce::
        File::userDesktopDirectory), "*");
    const auto fileFlags = juce::FileBrowserComponent::openMode
        ;

    chooser->launchAsync(fileFlags, [this](const juce::
        FileChooser& opened)
    {
        juce::File chosenFile (opened.getResult());
        // Code to handle chosen files data

        if (chosenFile.getFileExtension() == ".wav") {
            processorRef.savedFile = chosenFile;
            processorRef.irLoader.reset();
            processorRef.irLoader.loadImpulseResponse(
                chosenFile, juce::dsp::Convolution::Stereo::no,
                juce::dsp::Convolution::Trim::yes, 0);
        }

    });
}
```

References

[1] Barry Blesser and Linda-Ruth Salter. "Spaces speak, are you listening". In: *Experiencing aural architecture* 232 (2007) (page 4).

[2] F.M. Carrano and T. Henry. *Data Abstraction & Problem Solving with C++: Walls and Mirrors*. Pearson, 2016. ISBN: 9780134463971. URL: https://books.google.com/books?id=pVPfjgEACAAJ (page 33).

[3] *Comp.realtime FAQ*. 1998. URL: http://www.faqs.org/faqs/realtime-computing/faq/ (page 13).

[4] Mark Corless and Arvind Ananthan. "PC-Based Prototyping of Audio Applications Using Model-Based Design". In: *Journal of The Audio Engineering Society* 20 (June 2009) (page 13).

[5] Jon Dattorro. "Effect design, part 1: Reverberator and other filters". In: *Journal of the Audio Engineering Society* 45.9 (1997), pp. 660–684 (page 3).

[6] R.A. DeVoe. *Electronmusic: A Comprehensive Handbook*. Electronic Music Laboratories, 1977. URL: https://books.google.com/books?id=cM87AQAAIAAJ (page 6).

[7] Larson Davis: A PCB Division. *Reverberation Time Measurements: Understanding RT60 in Room Acoustics*. https://www.larsondavis.com/learn/building-acoustics/Reverberation-Time-in-Room-Acoustics. Accessed: 2025-03-09 (page 27).

[8] *Dual-Reverb*. https://github.com/ShotgunApe/Dual-Reverb. Accessed: 2025-03-15 (page 44).

[9] Angelo Farina et al. "Simultaneous measurement of impulse response and distortion with a swept-sine technique". In: *Preprints-Audio Engineering Society* (2000) (pages 29–30).

[10] Michael Gonzáles Harbour. "Real-time POSIX: An Overview". In: *VVConex 93 International Conference, Moscu* (June 1993) (page 14).

[11] David Griesinger. "Practical processors and programs for digital reverberation". In: *Audio Engineering Society Conference: 7th International Conference: Audio in Digital Times*. Audio Engineering Society. 1989 (page 23).

[12] TMP Associates Inc. and Bacik Karpinski Associates Inc. *Music & Fine Arts Building, The College of Wooster*. 1986 (page 28).

[13] jojeck. *Answer to "Calculating the inverse filter for the (exponential) sine sweep Method"*. https://dsp.stackexchange.com/questions/41696/calculating-the-inverse-filter-for-the-exponential-sine-sweep-method. Accessed: 2025-03-10; Licensed under CC BY-SA 3.0: https://creativecommons.org/licenses/by-sa/3.0/. 2017 (page 30).

[14] M. Tim Jones. "Anatomy of real-time Linux architectures". In: *IBM developerWorks* (Apr. 2008) (pages 13–14).

[15] *JUCE*. https://juce.com/. Accessed: 2025-02-14 (page 16).

[16] Dan Krites. Personal Communication. Employed as Assistant Director for Media Services, The College of Wooster. (page 30).

[17] Hammond Laurens. *Electrical musical instrument*. US Patent 2,230,836. Feb. 1941 (page 5).

[18] Samuel Ling, Jeff Sanny, and William Moebs. *University Physics, Volume 1*. OpenStax, 2016 (pages 9–11).

[19] Marshall Long. "8 - Sound in Enclosed Spaces". In: *Architectural Acoustics (Second Edition)*. Ed. by Marshall Long. Second Edition. Boston: Academic Press, 2014, pp. 313–344. ISBN: 978-0-12-398258-2. DOI: https://doi.org/10.1016/B978-0-12-398258-2.00008-8. URL: https://www.sciencedirect.com/science/article/pii/B9780123982582000088 (page 28).

[20] Geraint Luff. *Let's Write a Reverb*. https://signalsmith-audio.co.uk/writing/2021/lets-write-a-reverb/. Accessed: 2025-03-15 (page 43).

[21] "Model-View-Controller Pattern". In: *Learn Objective-C for Java Developers*. Berkeley, CA: Apress, 2009, pp. 353–402. ISBN: 978-1-4302-2370-2. DOI: 10.1007/978-1-4302-2370-2_20. URL: https://doi.org/10.1007/978-1-4302-2370-2_20 (page 16).

[22] Sophocles J. Orfanidis. *Introduction to Signal Processing*. Prentice Hall, 1998 (pages 11–12).

[23] Will Pirkle. *Designing audio effect plugins in C++: for AAX, AU, and VST3 with DSP theory*. Routledge, 2019 (pages 18, 23–26, 35).

[24] Melissa Puster. Personal Communication. Employed as Administrative Coordinator for the Music Department, The College of Wooster. (page 28).

[25] *Room EQ Wizard*. https://www.roomeqwizard.com/. Accessed: 2025-03-11 (page 30).

[26] Alessandro Rubini, Jonathan Corbet, and Greg Kroah-Hartman. *Linux Device Drivers*. en. 3rd ed. Sebastopol, CA: O'Reilly Media, Feb. 2005 (page 12).

[27] Sebastian J Schlecht and Emanuël AP Habets. "On lossless feedback delay networks". In: *IEEE Transactions on Signal Processing* 65.6 (2016), pp. 1554–1564 (page 43).

[28] Manfred R. Schroeder. "Natural Sounding Artificial Reverberation". In: *Journal of The Audio Engineering Society* 203 (Oct. 1961) (pages 5, 21–24).

[29] Manfred R. Schroeder and Benjamin F. Logan. ""Colorless" Artificial Reverberation". In: *IRE Transactions on Audio* 6 (1961), pp. 209–214 (page 21).

[30] Julius O. Smith. *Introduction to Digital Filters with Audio Applications*. W3K Publishing, 2007. ISBN: 978-0-9745607-1-7 (pages 20–21).

[31] Julius O. Smith. *Mathematics of the Discrete Fourier Transform (DFT)*. online book, 2007 edition. W3K Publishing, Accessed: 2025-02-16 (page 19).

[32] Julius O. Smith. *Physical Audio Signal Processing*. online book, 2010 edition. W3K Publishing, Accessed: 2025-02-25 (page 24).

[33] JCW Acoustic Supplies. *Absorption coefficients of common building materials and finishes*. https://www.acoustic-supplies.com/absorption-coefficient-chart/. Accessed: 2025-03-10 (pages 28–29).

[34] Pete Symons. *Digital Waveform Generation*. Cambridge University Press, 2013 (pages 10–11).

[35] Vesa Välimäki et al. "Fifty Years of Artificial Reverberation". In: *IEEE Transactions on Audio, Speech, and Language Processing* 20.5 (2012), pp. 1421–1448. DOI: 10.1109/TASL.2012.2189567 (pages 4–5, 23).

[36] Jezar Wakefield. *Free Studio Quality Reverb*. https://web.archive.org/web/20010627232434fw_/http://home.onet.co.uk/~jzracc/freeverb.htm. Accessed: 2025-02-26 (page 23).

[37] Martin Walker. *Optimising The Latency Of Your PC Audio Interface*. 2005. URL: https://www.soundonsound.com/techniques/optimising-latency-pc-audio-interface (pages 13–14).

[38] Jan Wilczek. *Circular vs. Linear Convolution: What's the Difference?* https://thewolfsound.com/circular-vs-linear-convolution-whats-the-difference/. Accessed: 2025-02-28 (page 25).